A Unified Model for Cardinality Estimation by Learning from Data and Queries via Sum-Product Networks

Abstract

Cardinality estimation is a fundamental component in database systems, crucial for generating efficient execution plans. Despite advancements in learning-based cardinality estimation, existing methods may struggle to simultaneously optimize the key criteria: estimation accuracy, inference time, and storage overhead, limiting their practical applicability in real-world database environments. This paper introduces QSPN, a *unified model* that integrates both data distribution and query workload. QSPN achieves high estimation accuracy by modeling data distribution using the simple yet effective Sum-Product Network (SPN) structure. To ensure low inference time and reduce storage overhead, QSPN further partitions columns based on query access patterns. We formalize QSPN as a tree-based structure that extends SPNs by introducing two new node types: QProduct and QSplit. This paper studies the research challenges of developing efficient algorithms for the offline construction and online computation of QSPN. And we also propose a novel multi-table cardinality estimation method M-QSPN by extending QSPN. We conduct extensive experiments to evaluate QSPN and M-QSPN on multiple single-table and multi-table datasets. The experimental results have demonstrated that QSPN achieves superior and robust performance on the three key criteria, compared with state-of-the-art approaches.

ACM Reference Format:

. 2026. A Unified Model for Cardinality Estimation by Learning from Data and Queries via Sum-Product Networks. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 16 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Cardinality estimation (CardEst), which estimates the result size of an SQL query on a relational database, is a fundamental component of query optimization in database management systems (DBMSs). Traditional CardEst methods [1, 2, 4, 10, 11, 18, 24] rely on simplifying assumptions, such as column independence, often leading to substantial estimation errors. To overcome these limitations, learning-based CardEst models [6, 12, 16, 20, 40, 41, 43] have emerged as state-of-the-art solutions, significantly improving accuracy by capturing data distributions and query patterns.

Despite the advancements in learning-based CardEst models, deploying them in real-world DBMS systems still requires balancing three key criteria [9, 15, 33, 34, 43]: *estimation accuracy*, *inference time*, and *storage overhead*. Data-driven approaches [12, 40, 41, 43] leverage probabilistic models, such as Sum-Product Networks [12, 20, 43] and Deep Auto-Regressive [40, 41], to capture the joint distribution of all columns of the relational data. While these methods typically achieve high estimation accuracy, they suffer from high inference time and substantial storage overheads, particularly when handling complex data distributions. Query-driven methods [6, 16], train regression models that directly map SQL queries to their cardinalities based on a set of training queries, bypassing the need to model data distributions. Although these methods are efficient and lightweight, they struggle with generalization, particularly when encountering queries that significantly deviate from those in the training set.

Given the strengths and limitations of existing methods, a promising direction is to develop a *unified model* that leverages both data and queries, aiming to achieve the three key criteria: *high estimation accuracy*, *low inference time*, and *lightweight storage overhead*. Such an approach can overcome the weaknesses of data-driven or query-driven CardEst models by leveraging complementary information from both sources.

Our Proposals. In this paper, we propose to learn from both data and queries via the Sum-Product Network (SPN) model. As shown in Figure 1(a), traditional SPN models [12, 43] recursively partition columns (*i.e.,* the Product node) and rows (*i.e.,* the Sum node) into *local subsets*, making it easier to compute the joint probability distribution from these local distributions. However, they often suffer from *high inference time* and *large model size* when columns in a database are highly correlated. This issue arises because many intermediate nodes (*e.g.,*Sum nodes) must be introduced to ensure that the columns in partitioned subsets are treated as independent.

To address this problem, we propose QSPN that extends SPNs by incorporating query workload information. The high-level idea of QSPN stems from an observation in many real-world workloads: queries often exhibit **specific access patterns** on the columns of relational tables, which can be leveraged to enhance both the efficiency and accuracy of CardEst. For instance, when retrieving movie comments by types, production year is usually a search criteria meanwhile *i.e.,* these two columns are frequently queried together, whereas company type is seldom focused together with type.

By integrating query workload information, QSPN can jointly partition columns based on both data correlations and query access patterns, thereby reducing model size and improving inference efficiency without sacrificing estimation accuracy. Table 1 presents a comparison between QSPN and existing approaches by the three key criteria mentioned above.

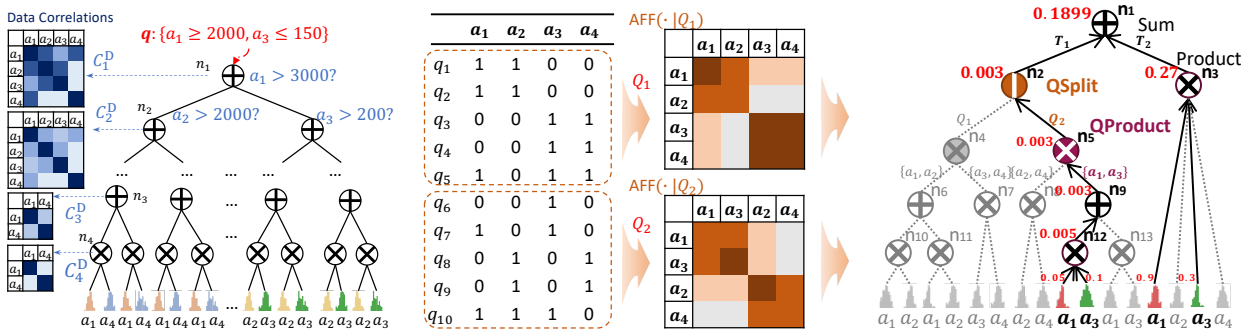
Example 1. *We consider the CardEst task for an example table T with highly correlated columns a_1, a_2, a_3, a_4, as illustrated in Figure 1(a). SPN partitions T into different row subsets via* Sum *nodes (*e.g., *node n_1, which partitions rows based on whether $a_1 > 3000$)*

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2026/06
https://doi.org/XXXXXXX.XXXXXXX



(a) Traditional SPN Model. (b) Partitioning Columns based on Query Patterns. (c) QSPN Model and CardEst Computation.

Figure 1: High-level idea of QSPN.

Table 1: Cardinality Estimation Methods Comparison

Method	Category	Accurate Estimation	Fast Inference	Lightweight Storage
Postgres	Traditional	☆	★★★	★★★
MHist	Traditional	☆	☆	★
Sampling	Traditional	★★	★★	★★★
MSCN [16]	Query-Driven	★★	★★★	★
Naru [41]	Data-Driven	★★★	★	★
DeepDB [12]	Data-Driven	★★	★★	★★
FLAT [43]	Data-Driven	★★★	★	☆
UAE [35]	Hybrid	★★★	★	★
Ours	**Hybrid**	★★★	★★★	★★★

to reduce column correlations within each subset. However, as depicted in the figure , when columns exhibit high correlations, the SPN requires numerous Sum *nodes to break down the joint distribution into local distributions over individual columns. This leads to a substantial increase in model size. Moreover, when processing a query q, the inference procedure must traverse a large number of these nodes, significantly increasing inference time.*

As illustrated in Figure 1(b), QSPN *leverages the* ***access patterns*** *of workload Q,* i.e., *how often certain columns are accessed together by the same queries. Within subset Q_1 of Q, we observe that columns a_1 and a_2 are frequently accessed together by queries q_1 and q_2, while a_3 and a_4 are jointly accessed by queries q_3 and q_4. Based on this pattern, we partition the columns into $\{a_1, a_2\}$ and $\{a_3, a_4\}$ for queries in Q_1, even if their data remain highly correlated. Similarly, for queries in subset Q_2, the columns can be partitioned into $\{a_1, a_3\}$ and $\{a_2, a_4\}$. These query-aware column partitions allow* QSPN *to construct more compact SPN models, reducing model size and inference time while maintaining accuracy.*

As shown in Figure 1(c), we formalize QSPN as a tree-based structure that extends SPNs by introducing two new node types: QProduct and QSplit. Specifically, QProduct partitions columns based on query-specific access patterns (*e.g.,* grouping frequently accessed columns together) within a given workload, while QSplit refines the workload itself into more granular subsets to capture workload variations. Moreover, QSPN retains the SPN's ability to partition columns and rows based on data correlations. By partitioning columns based on both data correlations and query access patterns, QSPN effectively reduces the number of intermediate nodes in the SPN, which improves inference efficiency, reduces storage overhead while maintaining high estimation accuracy.

Key Challenges and Solutions. We study the technical challenges that naturally arise in our proposed QSPN approach.

Offline QSPN *Construction.* A key challenge in offline QSPN construction is integrating query workload information into the SPN framework while maintaining inference efficiency. Unlike conventional SPNs, QSPN must consider query co-access patterns, making the partitioning problem more complex. To address this, QSPN develops efficient algorithms for two core problems, *i.e.,* QProduct for query-aware column partitioning and QSplit for workload partitioning.

Online QSPN *Computation.* The online stage of QSPN presents two key challenges. First, computing accurate cardinalities while minimizing inference overhead is non-trivial, especially for queries with unseen access patterns. Second, as data distributions and query workloads evolve, QSPN must maintain accuracy without frequent full retraining. To tackle these challenges, we develop an online inference algorithm, and introduce an incremental update mechanism that enables QSPN to efficiently adapt to workload and data changes.

Multi-Table CardEst with QSPN. Extending QSPN to multi-table cardinality estimation is challenging due to the complex distributions of join keys and the impact of base table query predicates. To address this, we introduce a novel approach M-QSPN based on QSPN for multi-table CardEst while maintaining high accuracy and efficient inference performance.

Contributions. Our contributions are summarized as follows.

(1) We propose QSPN, a query-aware Sum-Product Network that integrates data and queries for CardEst (Section 3).

(2) We develop effective algorithms for offline QSPN construction and online QSPN computation, balancing estimation accuracy, inference time, and model size (Sections 4 and 5). We propose M-QSPN supporting multi-table CardEst (Section 6).

(3) We conduct a comprehensive experimental study on widely used CardEst benchmarks. Extensive results demonstrate that QSPN achieves superior performance (Section 7).

2 Preliminaries

2.1 Problem Formalization

Data. This paper considers a relational table T with columns (attributes) $A = \{a_1, a_2, \ldots, a_{|A|}\}$ and tuples $T = \{t_1, t_2, \ldots, t_{|T|}\}$. We define the domain of column a_i as $[LB_i, UB_i]$, representing the lower and upper bounds of a_i.

Queries. Similar to existing works [12, 35, 43], this paper focuses on queries that consist of a *conjunction* of predicates, where each predicate over column a_i can be represented as $L_i \le a_i \le U_i$, with $LB_i \le L_i \le U_i \le UB_i$. Without loss of generality, the endpoints of interval $[L_i, U_i]$ can be open, *e.g.,* $(L_i, U_i]$ or $[L_i, U_i)$, which are omitted for simplicity. In particular, a point query over a_i can be represented as $L_i = U_i$. For ease of presentation, we assume each column has only one interval $[L_i, U_i]$, though this can be easily extended to the cases with multiple intervals per column.

This paper considers a *query workload* as a set of queries $Q = \{q_1, q_2, \ldots, q_{|Q|}\}$, typically extracted from real query logs. Given this workload Q, we introduce *query-column access matrix* (or *access matrix* for short) to represent the **access patterns** of queries in Q on columns in T. This matrix captures which queries reference which columns, enabling more optimization opportunities in query-aware CardEst.

Definition 2.1 (Query-Column Access Matrix). For each query q_i and each column a_j, we define $\texttt{acc}(q_i, a_j)$ as an indicator function that specifies whether column a_j is accessed by query q_i. Specifically, $\texttt{acc}(q_i, a_j) = 1$ if q_i accesses a_j, and $\texttt{acc}(q_i, a_j) = 0$ otherwise. Then, we define the *access matrix* for a workload Q and a column set A, denoted as $\mathbf{ACC}(Q, A)$, as a binary matrix where each entry $\texttt{acc}(q_i, a_j)$ indicates whether query q_i accesses column a_j.

In particular, we use $\mathbf{acc}(q_i)$ to denote the i-th row of access matrix, representing the access pattern of query q_i across all columns, and $\mathbf{acc}(a_j)$ to denote the j-th column of access matrix, capturing how column a_j is accessed by queries.

CardEst. Given a new query q, we define $\texttt{Card}(q, T)$ as the *cardinality* of q over table T, *i.e.,* the number of tuples in T that satisfy the query conditions. The goal of *cardinality estimation* is to compute an estimate $\widehat{\texttt{Card}}(q, T)$ that approximates $\texttt{Card}(q, T)$ efficiently and accurately without executing q on T. Specifically, we study the following problems: (1) *Offline CardEst Training*, which trains a CardEst model by leveraging both data T and workload Q, capturing the underlying data distribution and query access patterns, and (2) *Online CardEst Inference*: which uses the trained CardEst model to estimate the cardinality $\widehat{\texttt{Card}}(q, T)$ for a given query q.

2.2 SUM-Product Network (SPN) Model

Sum-Product Network (SPN) [23] is a data-driven model with extensions such as DeepDB [12] and FLAT [43]. SPN-based approaches address the CardEst problem by modeling the *joint probability distribution* $P_T(A)$, where $\forall a_i \in A$ is treated as a random variable. Given a query q with predicates $a_i \in [L_i, U_i]_{i=1}^m$, the cardinality is estimated as $\widehat{\texttt{Card}}(q, T) = |T| \cdot \sum_{v_1 \in [L_1, U_1]} \cdots \sum_{v_m \in [L_m, U_m]} P_T(v_1, \ldots, v_m)$, where the summation iterates over all possible values within the query's range constraints. SPN approximates $P_T(A)$ by *decomposing* the joint probability distribution into multiple *local probability distributions*. This decomposition is realized through a hierarchical, tree-based structure, where each node represents a *local* joint probability distribution $P_{T'}(A')$.

The key idea of SPN focuses on introducing intermediate nodes, which fall into one of the following two categories. (1) A `Sum` node partitions its tuple set T' into a collection of disjoint subsets $T' = \bigcup_i T'_i$. Each subset T'_i corresponds to a child node with a probability distribution $P_{T'_i}(A')$. The overall distribution at the `Sum` node is then computed as a weighted sum of its children's distributions $P_{T'}(A') = \sum_i w_i \cdot P_{T'_i}(A')$, where the weight w_i is determined by the proportion of tuples in each subset, given by $w_i = |T'_i|/|T'|$. (2) A `Product` node partitions its column set A' into disjoint subsets $A' = \bigcup_j A'_j$. Each subset A'_j corresponds to a child node that models the probability distribution $P_{T'}(A'_j)$. By assuming independence among these subsets, the overall distribution at the `Product` node is computed as $P_{T'}(A') = \prod_j P_{T'}(A'_j)$. This decomposition allows SPNs to efficiently approximate complex joint distributions by leveraging independence between columns in a particular data subset. Given a query q for CardEst, SPN estimates the cardinality $\widehat{\texttt{Card}}(q, T)$ in a bottom-up manner.

3 An Overview of QSPN

We propose `QSPN` that extends traditional SPNs by incorporating query workload information to partition columns based on their *access patterns*. We formalize `QSPN` as a tree-based structure that extends traditional SPNs by introducing two new node types, `QProduct` and `QSplit`, as shown in Figure 1(c). Formally, each node $\texttt{n}$ in `QSPN` is represented as a 4-tuple $(A_\texttt{n}, T_\texttt{n}, Q_\texttt{n}, O_\texttt{n})$, where $A_\texttt{n}$ denotes the column set, $T_\texttt{n}$ the corresponding table, and $Q_\texttt{n}$ the associated query workload. Each node captures the joint probability distribution $P_{T_\texttt{n}}(A_\texttt{n})$ conditioned on the queries in $Q_\texttt{n}$. Moreover, the node type $O_\texttt{n}$ represents how the joint probability of node $\texttt{n}$ is estimated from its child nodes, which is described as follows.

`QProduct`. A `QProduct` node, such as $\texttt{n}_5$ in Figure 1(c), partitions its column set $A_\texttt{n}$ into a set of disjoint subsets $\mathcal{A} = \{A_1, \ldots, A_m\}$, ensuring that columns in different subsets are infrequently co-accessed by queries in $Q_\texttt{n}$. For each subset A_i, a corresponding child node is created as $\texttt{n.child}_i = (A_i, T_\texttt{n}[A_i], Q_\texttt{n}[A_i], O_i)$. The joint probability distribution $P_{T_\texttt{n}}(A_\texttt{n})$ at node $\texttt{n}$ is computed as $P_{T_\texttt{n}}(A_\texttt{n}) = \prod_{i=1}^m P_{T_\texttt{n}}(A_i)$.

`QSplit`. A `QSplit`, such as $\texttt{n}_2$ in Figure 1(c), splits its query workload $Q_\texttt{n}$ into a set of disjoint query subsets $\boldsymbol{Q} = \{Q_1, \ldots, Q_m\}$, ensuring that queries within the same subset share similar access patterns, while queries across different subsets exhibit distinct access behaviors. For each query subset Q_i, a corresponding child node is created as $\texttt{n.child}_i = (A_\texttt{n}, T_\texttt{n}, Q_i, O_i)$. Note that a `QSplit` does not directly compute a joint probability distribution $P_{T_\texttt{n}}(A_\texttt{n})$ but instead functions as a *query router*. Specifically, when estimating the cardinality of a query q, the `QSplit` identifies the query subset Q_{k^*} from $\boldsymbol{Q}$ that shares the most *similar access patterns* with q and routes q to the corresponding child node $\texttt{n.child}_{k^*}$ for cardinality estimation. We will discuss the method for determining whether a query subset exhibits the most *similar access patterns* with q later.

`Product`. A `Product` node, such as $\texttt{n}_3$ in Figure 1(c), partitions its column set $A_\texttt{n}$ into a set of disjoint subsets $\mathcal{A} = \{A_1, \ldots, A_m\}$,

ensuring statistical independence between columns in different subsets. For each subset A_i, a corresponding child node is created as $\mathsf{n.child}_i = (A_i, T_\mathsf{n}[A_i], Q_\mathsf{n}[A_i], O_i)$. The joint distribution $P_{T_\mathsf{n}}(A_\mathsf{n})$ is computed using the equation in Section 2.2.

Sum. A Sum node, such as n_1 in Figure 1(c), partitions the table T_n of node n into disjoint subsets $\mathcal{T} = \{T_1, \ldots, T_m\}$. For each subset T_i, a corresponding child node is created as $\mathsf{n.child}_i = (A_\mathsf{n}, T_i, Q_\mathsf{n}, O_i)$. The joint distribution $P_{T_\mathsf{n}}(A_\mathsf{n})$ is computed using the equation in Section 2.2.

Leaf. A Leaf node, such as any leaf of the tree in Figure 1(c), represents the 1-dimensional probability distribution $P_{T_\mathsf{n}}(A_\mathsf{n})$. Specifically, we use a histogram-based mechanism to capture it. The construction cost of the histogram is $O(|T_\mathsf{n}|)$, and the query cost is approximately $O(1)$.

To support the above QSPN structure, we introduce a framework that consists of both offline and online stages.

Offline QSPN Construction. In the offline stage, QSPN learns its structure from both data T and workload Q, capturing the underlying data distribution and query access patterns. Unlike conventional SPNs, QSPN introduces a new challenge of incorporating query co-access patterns into column partitioning. We propose efficient algorithms QProduct and QSplit, as presented in Section 4.

Online QSPN Computation. In the online stage, we utilize QSPN for CardEst. Specifically, for queries with access patterns that differ from those in the training workload, we develop an online inference algorithm, ensuring both accuracy and efficiency. Second, we introduce an incremental update mechanism that selectively identifies and updates only the affected parts of the model. Details of online QSPN inference are provided in Section 5.

Multi-Table CardEst. This paper also explores extending QSPN to support multi-table CardEst. The key difficulty lies in accurately modeling join key distributions while effectively handling multi-table query predicates. Traditional methods rely on heuristic bucket-based approaches, which suffer from poor accuracy. Learning-based CardEst models [40, 43], on the other hand, train on a materialized outer-join table, incurs excessive time and storage costs. To tackle this, we introduce an effective approach, which is described in Section 6.

4 Offline QSPN Construction

Given a table T with column set A and a workload Q, QSPN construction generates a QSPN tree that models the joint probability distribution $P_T(A)$ conditioned on Q. To achieve this, the construction process recursively decomposes the joint probability distribution into local probability distributions in a top-down manner. Specifically, during the construction of each node, QSPN attempts different node types in the following order: Leaf, Product, QProduct, QSplit, and Sum.

Construction of Leaf Nodes. During the recursive process, if $|A| = 1$, this indicates that the joint probability distribution has been fully decomposed into a local distribution over the specific column. In this case, the construction process creates a Leaf to model the 1-D probability distribution $P_{T_n}(A)$ using a histogram. This choice is motivated by the histogram's accuracy, efficiency, and lightweight nature, making it well-suited for modeling such distributions.

Construction of Product Nodes. A Product node is constructed when the column set A of a node exhibits statistical dependencies suitable for partitioning. Following prior works [12, 23, 43], we use the *Randomized Dependence Coefficient* (RDC) [19] to measure statistical dependencies between columns. Then, we employ a partition-based method. Specifically, we construct a graph where vertices represent columns and edges represent dependencies weighted by the RDC values. Then, we remove the edges with RDC values below a threshold, and divide the graph into connected components, each representing an independent subset of columns.

Construction of Sum Nodes. During the recursive process, if all other types of nodes fail to meet the decomposition criteria, node n defaults to a Sum, which splits T into subsets T_i. For this, following prior works [12, 43], we use the K-Means clustering algorithm, as it partitions the data into clusters, which helps to reduce data correlation within each subset.

4.1 Construction of QProduct

QProduct partitions columns according to their query access patterns, grouping frequently co-accessed columns together while separating those that are rarely co-accessed into distinct subsets.

Formalization of QProduct. We first formally define *access affinity* using the query-column access matrix as follows.

Definition 4.1 (Access Affinity). The *Access Affinity* between columns a_i and a_j with respect to query workload Q, denoted by $\mathtt{AFF}(a_i, a_j|Q)$, is defined as how frequently both columns are referenced together by queries in Q, *i.e.,*

$$\mathtt{AFF}(a_i, a_j|Q) = \mathbf{acc}(a_i) \cdot \mathbf{acc}(a_j). \tag{1}$$

We define the QProduct operation using access affinity.

*Definition 4.2 (*QProduct*).* Given a workload Q and a column set $A = \{a_1, \ldots, a_{|A|}\}$, QProduct partitions A into a set of disjoint column subsets $\mathcal{A} = \{A_1, \ldots, A_m\}$, minimizing the *inter-partition affinity* (*IPA*) of partitioning $\mathtt{IPA}(\mathcal{A}|Q)$, where

$$\mathtt{IPA}(\mathcal{A}|Q) = \sum_{1 \le k < l \le m} \sum_{a_i \in A_k} \sum_{a_j \in A_l} \mathtt{AFF}(a_i, a_j|Q). \tag{2}$$

For example, consider the workload Q_1 in Figure 1(b). The inter-partition affinity for the column partitioning $\{\{a_1, a_2\}, \{a_3, a_4\}\}$ is 0, whereas for the partitioning $\{\{a_1, a_3\}, \{a_2, a_4\}\}$, it is 4. Based on these results, QProduct selects the partition $\{\{a_1, a_2\}, \{a_3, a_4\}\}$ for workload Q_1 to minimize inter-partition affinity.

Algorithm Design for QProduct. We first analyze the complexity of the QProduct construction problem.

Lemma 1. *The problem of* QProduct *construction is equivalent to the minimum k-cut problem.*

Proof. Given an arbitrary undirected graph $G = (V, E)$ and parameter K, we make a column set $A = \{a_1, \ldots, a_{|V|}\}$ corresponding to V and we set $\mathtt{AFF}(a_i, a_j|Q) = \mathtt{AFF}(a_j, a_i|Q) = w_l / \max\{w|e = (u, v, w) \in E\}$ for each edge $e_l = (v_i, v_j, w_l)$. Note that the AFF matrix we set must be legal since the assumed query workload Q which contains $\mathtt{AFF}(a_i, a_j|Q) * \max\{w|e = (u, v, w) \in E\}$ queries

with predicates only on column a_i and a_j for each $\texttt{AFF}(a_i, a_j|Q)$ value is easy to generate. Then we solve QProduct construction problem to partition A into K subsets: $A = \{A_1, \ldots, A_K\}$ and we get the answer of the K-CUT problem: minimized $IPA(A|Q)$ multiplying $\max\{w|e = (u, v, w) \in E\}$. □

The minimum k-cut problem, even for fixed k, is computationally expensive to solve. We abstract each column to a vertice. For example, the minimum 2-cut can be solved in $O(|A|^3)$ time using algorithms such as Stoer-Wagner [30]. For $k = 3$, the time complexity is $O(|A|^3\tau(|A|))$ [22], where $\tau(|A|)$ represents the cost of computing the objective function, which is $O(|A|^2)$ in our QProduct construction problem, yielding an overall complexity of $O(|A|^5)$. Similarly, a recent algorithm for $k = 4$ achieves a complexity of $O(|A|^6\tau(|A|))$ (or $O(|A|^8)$). While the problem is polynomial-time solvable for fixed $k \geq 5$, the complexity increases dramatically (*e.g.,* $O(|A|^{16})$) for a minimum 5-cut algorithm as suggested by [13]), rendering such algorithms impractical for real-world use. Even a common approximation algorithm based on Gomory-Hu Tree is too slow ($O(|A|^5)$) for construction with many QProduct tries.

Given the computational expense of partitioning the column set to minimize IPA, we design an algorithm **PartitionByAFF** that achieves effective and efficient ($O(|A|^2)$) results. The algorithm first constructs a graph $G = (A, E)$, where vertices A represent columns, and an edge $e_{ij} \in E$ connects a_i and a_j if their affinity $\texttt{AFF}(a_i, a_j|Q)$ exceeds a threshold τ. Next, the algorithm identifies the connected components of G, with the vertices in each connected component $G_i \subseteq G$ forming a column partition A_i.

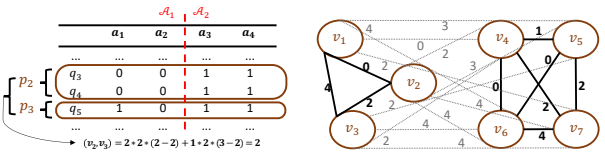
4.2 Construction of QSplit

When considering the entire workload Q, QProduct may struggle to derive a meaningful column partitioning with a sufficiently low IPA score due to the presence of queries exhibiting diverse access patterns. The following example illustrates this challenge.

Example 2. *Given the workload $Q = \{q_1, q_2, \ldots, q_{10}\}$, as shown in Figure 1(b), an optimal column partitioning for* QProduct *with $k = 2$ results in $\{\{a_1, a_2, a_3\}, \{a_4\}\}$, yielding a minimum IPA score of 6, which remains relatively high. The primary reason for this is that different subsets of queries within Q exhibit different access patterns, leading to conflicting preferences for column partitioning. Specifically, queries in $Q_1 \subset Q$ favor the column partitioning $\{\{a_1, a_2\}, \{a_3, a_4\}\}$, while those in $Q_2 \subset Q$ prefer $\{\{a_1, a_3\}, \{a_2, a_4\}\}$. These conflicting preferences arise due to the distinct access patterns exhibited by queries in different subsets.*

Formalization of QSplit. To address this challenge, we introduce the QSplit operation, which partitions the workload Q into n subsets, $\mathbf{Q} = \{Q_1, \ldots, Q_n\}$, ensuring that each subset exhibits more consistent access patterns, thus enabling QProduct to derive more meaningful column partitions.

*Definition 4.3 (*QSplit*).* Given a query workload Q and a column set $A = \{a_1, a_2, \ldots, a_{|A|}\}$, QSplit aims to partition Q into a set of disjoint query subsets $\mathbf{Q} = \{Q_1, Q_2, \ldots, Q_n\}$, minimizing the objective $\sum_{k=1}^{n} \texttt{IPA}(\mathcal{A}_k^*|Q_k)$, where $\mathcal{A}_k^*$ is the *optimal* column partition given query subset Q_k.



(a) Upper bound design. (b) Our greedy algorithm.

Figure 2: An example of QSplit construction.

For example, by splitting Q in Figure 1(b) into two subsets ($n = 2$), an effective partitioning results in $Q_1 = \{q_1, \ldots, q_5\}$ and $Q_2 = \{q_6, \ldots, q_{10}\}$. Given these subsets, the optimal QProduct column partitioning for Q_1 is $\mathcal{A}_1^* = \{\{a_1, a_2\}, \{a_3, a_4\}\}$ with $\texttt{IPA}(\mathcal{A}_1^*|Q_1) = 2$ and that for Q_2 is $\mathcal{A}_2^* = \{\{a_1, a_3\}, \{a_2, a_4\}\}$ with $\texttt{IPA}(\mathcal{A}_2^*|Q_2) = 2$.

Algorithm Design for QSplit. The QSplit construction problem is highly challenging because it requires simultaneously minimizing $\texttt{IPA}(\mathcal{A}_k^*|Q_k)$ for each partitioned subset Q_k. Thus, instead of directly optimizing $\texttt{IPA}(\mathcal{A}_k^*|Q_k)$ for each Q_k, we focus on minimizing its upper bound, denoted as $\overline{\texttt{IPA}}(\mathcal{A}_k|Q_k)$, which allows us to design a more tractable algorithm while still ensuring effective workload partitioning. Formally, the objective is to partition Q into $\mathbf{Q} = \{Q_1, Q_2, \ldots, Q_n\}$ while minimizing the upper bound of inter-partition affinity, *i.e.,* $\sum_{k=1}^{n} \overline{\texttt{IPA}}(\mathcal{A}_k|Q_k)$.

Next, we first design a mechanism to construct the upper bound $\overline{\texttt{IPA}}(\mathcal{A}_k|Q_k)$ and prove that optimizing this upper bound is NP-hard. Finally, we propose an efficient heuristic algorithm that achieves effective partitioning results.

Upper-bound design. Given a specific query set Q, we construct an upper bound $\overline{\texttt{IPA}}(\mathcal{A}|Q)$ for the optimal column partitioning result $\texttt{IPA}(\mathcal{A}^*|Q)$ by analyzing the access patterns of queries in Q. Since different queries (*e.g.,* q_1 and q_2 in Figure 1(b)) may share the same access pattern (e.g., $(1, 1, 0, 0)$), we introduce p_i to denote the i-th distinct access pattern in Q and let n_i represent the number of queries in Q that follow pattern p_i. For instance, in Figure 1(b), the first access pattern in Q is $p_1 = (1, 1, 0, 0)$, which corresponds to two queries, *i.e.,* $n_1 = 2$. Based on these notations, we construct the following upper bound for $\texttt{IPA}(\mathcal{A}^*|Q)$.

Definition 4.4 (Upper Bound). Given a Q consisting of m distinct query patterns $\{p_1, \ldots, p_m\}$, let $\|p_i\|$ denote the L_2 norm of pattern p_i, n_i represent the number of queries corresponding to pattern p_i, and z_{ij} denote the dot product of two patterns p_i and p_j. An upper bound for $\texttt{IPA}(\mathcal{A}^*|Q)$:

$$\overline{\texttt{IPA}}(\mathcal{A}|Q) = \sum_{i<j} n_i \cdot z_{ij} \cdot (\|p_i\| - z_{ij}) + n_j \cdot z_{ij} \cdot (\|p_j\| - z_{ij}).$$

Consider a Q with two query patterns (e.g., p_2 and p_3 in Figure 2(a)). To compute $\overline{\texttt{IPA}}(\mathcal{A}|Q)$, we calculate the dot product of p_2 and p_3: $z_{23} = p_2 \cdot p_3 = 2$, as well as the norms $\|p_2\| = 2$ and $\|p_3\| = 3$. Given that $n_2 = 2$ and $n_3 = 1$, the contribution of p_2 and p_3 to the overall upper bound is $n_2 \cdot z_{23} \cdot (\|p_2\| - z_{23}) + n_3 \cdot z_{23} \cdot (\|p_3\| - z_{23}) = 2$. As shown in Figure 2(a), the upper bound corresponds to a specific column partitioning strategy, *i.e.,* grouping columns that are co-accessed by multiple query patterns (*e.g.,* A_3 and A_2) while placing the remaining columns (*e.g.,* A_1 and A_4) in a separate group.

Lemma 2. *For any given query set Q, $\overline{\texttt{IPA}}(\mathcal{A}|Q)$ provides an upper estimate of the optimal result $\texttt{IPA}(\mathcal{A}^*|Q)$.*

Proof. $\texttt{IPA}(\mathcal{A}^*|Q)$ can be decomposed into the IPA scores of distinct query patterns, *i.e.,* $\texttt{IPA}(\mathcal{A}^*|Q) = \sum_{p_i \in Q} \texttt{IPA}(\mathcal{A}^*|\{p_i\})$. For any query pattern $p_i \in Q$, its optimal IPA score $\texttt{IPA}(\mathcal{A}^*|\{p_i\})$ is bounded by the upper term $n_i \cdot z_{ij} \cdot (\|p_i\| - z_{ij})$, where p_j is any query pattern satisfying $z_{ij} > 0$. Since z_{ij} represents the number of shared accessed columns between patterns p_i and p_j, the term $(\|p_i\| - z_{ij})$ captures the remaining accessed columns in p_i that are not shared with p_j. Thus, $n_i \cdot z_{ij} \cdot (\|p_i\| - z_{ij})$ is the IPA score of the specific partitioning scheme, which is larger than the optimal result $\texttt{IPA}(\mathcal{A}^*|\{p_i\})$. On the other hand, if $z_{ij} = 0$ for all $j \neq i$, p_i shares no common accessed columns with any other query patterns, then the optimal column partition for p_i incurs no IPA cost, implying $\texttt{IPA}(\mathcal{A}^*|\{p_i\}) = 0$. □

Hardness of upper-bound optimization. Next, we show that even optimizing $\overline{\texttt{IPA}}(\mathcal{A}|Q)$ is theoretically intractable.

Lemma 3. *The problem of partitioning Q into $\mathbf{Q} = \{Q_1, Q_2, \ldots, Q_n\}$ to minimize the upper bound of inter-partition affinity,* i.e.,, $\sum_{k=1}^{n} \overline{\texttt{IPA}}(\mathcal{A}_k|Q_k)$*, is NP-hard.*

Proof. We reduce this lemma from the Max-Cut problem. To convert an arbitrary graph $G(V, E)$ to QSplit problem, we generate $\mathbf{q_i}$ corresponding to each $v_i \in V$ and we generate $w = |\mathbf{q_i}|z_{i,j}(x_i - z_{i,j}) + |\mathbf{q_j}|z_{i,j}(x_j - z_{i,j})$ for each $e_{ij} \in E$. To maximize the sum of edge weights that are cut off in G, the sum of edge weights remained in all components is minimized, which is equivalent to minimizing $\sum_{k=1}^{n} \overline{\texttt{IPA}}(\mathcal{A}_k|Q_k)$ after converting G to QSplit problem. □

A greedy algorithm for QSplit. Given the time complexity of solving our problem, we design a practical and efficient heuristic algorithm. The algorithm first constructs a graph $G = (V, E)$, where each vertex $v_i \in V$ represents a query pattern p_i, and an edge $e_{ij} \in E$ connects two patterns p_i and p_j with a weight of $n_i \cdot z_{ij} \cdot (\|p_i\| - z_{ij}) + n_j \cdot z_{ij} \cdot (\|p_j\| - z_{ij})$. Next, based on the constructed graph G, the algorithm aims to partition it into n sub-graphs, denoted as $\{G_1, G_2, \ldots, G_n\}$, such that the total weight of the edges *across* the sub-graphs is maximized. To achieve this, it first sorts all vertices in G based on their weighted degree in descending order and initializes n empty sub-graphs. Then, the algorithm iteratively processes each vertex and assigns it to an *appropriate* sub-graph. Specifically, in the i-th iteration, the algorithm considers vertex v_i and evaluates its potential assignment to each sub-graph G_j by computing the total edge weight of G'_j after incorporating v_i. The vertex v_i is then assigned to the sub-graph that results in the minimum weight summation. Figure 2(b) illustrates an example of our greedy algorithm partitioning the query set Q from Figure 1(b) into two subsets (*i.e.,* $m = 2$).

5 Online QSPN Computation

5.1 CardEst Inference with QSPN

When a new query q arrives, the Online CardEst Inference process using QSPN operates recursively by traversing the QSPN tree from the root to the leaf nodes, as shown in Figure 1(c). Specifically, during traversal, if a QSplit is visited, the process routes query q to the child node corresponding to the most relevant query subset. On the other hand, if a QProduct, Product, or Sum is visited, the process computes the joint probability accordingly as follows.

(1) For a Leaf node n, the algorithm computes the probability $\texttt{n}.P_T(q)$ based on the histogram at leaf n.

(2) For a QProduct or Product node, n, the algorithm recursively invokes the CardEst inference process for each child of n where $q.A \cap \texttt{n.child}_i.A \neq \emptyset$. It then multiplies the estimated probabilities of all the relevant child nodes to produce the estimation result. For example, at QProduct node $\texttt{n}_5$, the column set is partitioned into a_1, a_3 and a_2, a_4. Since q involves only a_1, a_3, the estimation continues with node $\texttt{n}_9$, while node $\texttt{n}_8$ is pruned.

(3) For a Sum node, n, the algorithm computes the weighted sum of the estimated probabilities of its child nodes.

Next, we explore two key implementation details of the CardEst Inference process: (1) query routing in QSplit nodes, and (2) subtree pruning in Sum and Product nodes.

Query Routing in QSplit Nodes. The objective of query routing in a QSplit node (say $\texttt{n}_2$ in Figure 1(c)) is to measure the degree to which a query set Q shares *similar access patterns* with a given query q, which guides the QSplit node in routing q to the appropriate child node (*e.g.,* node $\texttt{n}_5$). To this end, we formally introduce a matching score $\texttt{S}(Q, q)$ between a query set Q and a query q:

$$\texttt{S}(Q, q) = \frac{1}{|Q|} \sum_{(a_i, a_j), i<j} \texttt{AFF}(a_i, a_j|Q) \cdot \texttt{AFF}(a_i, a_j|\{q\}) \quad (3)$$

The intuition behind the matching score $\texttt{S}(Q, q)$ is to measure how closely the access patterns of queries in Q align with the access pattern of q. For example, consider the query q executed in Figure 1(c). The access pattern for q is:

$$\texttt{AFF}(\cdot, \cdot|\{q\}) = [[1, 0, 1, 0], [0, 0, 0, 0], [1, 0, 1, 0], [0, 0, 0, 0]].$$

Considering the workload partitioning in Figure 1(b), we have $\texttt{AFF}(\cdot, \cdot|Q_1) = [[3, 2, 1, 1], [2, 2, 0, 0], [1, 0, 3, 3], [1, 0, 3, 3]]$ and $\texttt{AFF}(\cdot, \cdot|Q_2) = [[2, 1, 2, 0], [1, 3, 1, 2], [2, 1, 3, 0], [0, 2, 0, 2]]$ for Q_1 and Q_2, respectively. So we compute $\texttt{S}(Q_1, q) = \frac{1}{5}$ and $\texttt{S}(Q_2, q) = \frac{2}{5}$. Therefore, when the node $\texttt{n}_1$ is visited, the algorithm routes query q to its child node $\texttt{n}_5$, as Q_2 shares more similar access patterns with q than Q_1.

Pruning Rules in Sum and Product Nodes. We employ pruning rules that leverage query q and pre-computed metadata to exclude irrelevant child nodes of a given node n. (1) For Product and QProduct nodes, let A_q denote the set of columns constrained by the query q. For a child node $\texttt{n.child}_k$ of n, corresponding to the column subset A_k, pruning occurs if $A_k \cap A_q = \emptyset$. (2) For Sum nodes, it decides which child nodes contributes to $n.P_T(q)$ so that participate the computation *i.e.,* set of visited child nodes C. Consider a child node $\texttt{n.child}_k$ of n, corresponding to the table subset T_k. In offline construction, we pre-compute and store the range of values for each column A_i in T_k. During online CardEst, if the value ranges of T_k for any column do not overlap with the constraints specified by query q, the child node $\texttt{n.child}_k$ can be safely pruned, as it does not contribute to the result.

5.2 QSPN Model Update

Data updates (ΔT, which include new tuples) and query workload shifts (ΔQ, which include new queries) impact the accuracy and inference efficiency of the original QSPN model. The high-level idea

of the update method is to traverse QSPN in a top-down manner. Each time a node n is visited during the traversal, two steps are performed to update the subtree rooted at n. First, the method examines whether n, originally constructed using $n.T$ and $n.Q$, still fits $n.T \cup \Delta T$ or $n.Q \cup \Delta Q$. Second, if n no longer fits, the subtree rooted at n is reconstructed by calling the QSPN construction method (see Section 4), which generates a new subtree rooted at n′. Otherwise, each child $n.child_i$ is recursively updated. Note that the check and reconstruction steps are unnecessary for Leaf as histograms can be incrementally updated in $O(|\Delta T|)$.

The key challenge in the above update method is efficiently examining whether a node n still fits $n.T \cup \Delta T$ or $n.Q \cup \Delta Q$, as the corresponding data table $n.T$ and query workload $n.Q$ are not materialized at node n. To address this challenge, we maintain lightweight data structures in different types of nodes and design a mechanism to check whether node n is still up-to-date with respect to the data and query workload:

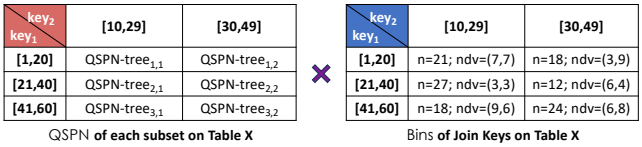
(1) If n is a Product node, we examine whether the column partitioning still holds, *i.e.,* whether columns in different partitions remain independent with respect to the updated data table $T \cup \Delta T$. To do this, we first compute $\mathtt{RDC}(a_i, a_j|\Delta T)$, where $a_i, a_j \in n.A$, and then check whether there exist a_i, a_j from different child nodes, say a_i from $n.child_k$ and a_j from $n.child_l$ such that $\frac{|n.T|}{|n.T|+|\Delta T|}\mathtt{RDC}(a_i, a_j|n.T) + \frac{|\Delta T|}{|n.T|+|\Delta T|}\mathtt{RDC}(a_i, a_j|\Delta T)$ is larger than a pre-defined threshold. If any such pair a_i, a_j is found, the subtree rooted at n needs reconstructing for more accurate CardEst. If the RDC between columns within a child node becomes less significant due to ΔT, we may reconstruct the subtree rooted at the child node to further partition the now-independent columns.

(2) If n is a QProduct node, the update examination strategy is similar to that of the Product case, except that we consider the access affinity $\mathtt{AFF}(a_i, a_j|n.Q)$ for any column pair (a_i, a_j), instead of the correlation $\mathtt{RDC}(a_i, a_j|n.T)$.

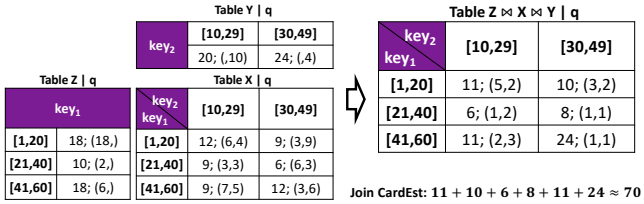
(3) If n is a QSplit, we examine whether the query routing strategy still holds for the updated workload $Q \cup \Delta Q$. To do this, for each workload partition Q_k corresponding to the child node $n.child_k$, we maintain the average of the matching scores of the queries in Q routed to Q_k, i.e., $\sum_{q \in Q_k} \mathtt{S}(Q_k, q)/|Q_k|$. Then, for each query q' in ΔQ, we assign q' to the child node with the maximum matching score (see Section 5.1) and update the average matching score accordingly. If the average matching score of any workload partition becomes less significant, *e.g.,* less than a predefined threshold, we reconstruct the subtree rooted at n, as the workload partition no longer reflects the access patterns of $Q \cup \Delta Q$.

(4) If n is a Sum node, we maintain the centroid for each tuple subset T_i from $\mathcal{T} = \{T_1, T_2, \ldots, T_m\}$ and the average distance between each tuple and the centroid of its assigned subset. Then, for each new tuple t in ΔT, we assign t to the tuple subset with the minimum distance to the centroid of that subset and update the average distance accordingly. If the average distance becomes significant, *e.g.,* exceeding a predefined threshold, we update the subtree rooted at n, as the Sum may no longer hold for $n.T \cup \Delta T$.

In this way, the QSPN update method minimizes unnecessary cost on model updates while ensuring accuracy in response to data updates and query workload shifts.



(a) QSPN for Base Table Filtering and Binning of Join Keys.



(b) Multi-Table CardEst with Star Join on 2 Groups of Join Keys

Figure 3: Illustration of our proposed M-QSPN method.

6 Multi-Table CardEst by M-QSPN

We introduce M-QSPN, a multi-table CardEst method built upon QSPN. For clarity, this section firstly considers a query q that performs a star join on two groups of join keys: key_1, key_2 among three tables X, Y and Z. In query q, table Y and Z are inner joined to the *center table* X on the condition $X.key_1 = Z.key_1 \wedge X.key_2 = Y.key_2$, denoted as $q(Z \bowtie X \bowtie Y)$. We assume that join keys in the same group share the same value domain *i.e.,* $X.key_1$ and $Z.key_1$ share domain D_1; $X.key_2$ and $Y.key_2$ share domain D_2. Besides, query q has base table filter predicates $q(X)$, $q(Y)$ and $q(Z)$. Thus, the CardEst of star join queries (say q) can be formalized as estimating $|q(Z \bowtie X \bowtie Y)| = |X||Y||Z| \sum_{v_1 \in D_1} \sum_{v_2 \in D_2} P(X.key_1 = v_1 \wedge X.key_2 = v_2 \wedge q(X)) \cdot P(Z.key_1 = v_1 \wedge q(Z)) \cdot P(Y.key_2 = v_2 \wedge q(Y))$ where $P(X.key_1 = v_1 \wedge X.key_2 = v_2 \wedge q(X))$ denotes the probability that the join key $X.key_1$ and $X.key_2$ equals v_1 and v_2 respectively in the result table of the base filter predicates $q(X)$. It is similar for $P(Z.key_1 = v_1 \wedge q(Z))$ (or $P(Y.key_2 = v_2 \wedge q(Y))$).

As directly estimating $|q(Z \bowtie X \bowtie Y)|$ is computationally expensive, M-QSPN supports multi-table CardEst by *binning join keys*: dividing the domains of join keys in equal-width way into a set of **bins** which are organized to a k-dimensional grid for a table involving k groups of join keys. Specifically, on table X, D_1 and D_2 of the join keys key_1 and key_2 are divided into a set of **bins**: $\mathbf{Bins}_X = \{\mathtt{Bin}_{1,1}, \ldots, \mathtt{Bin}_{1,n_2}, \mathtt{Bin}_{2,1}, \ldots, \mathtt{Bin}_{n_1,1}, \ldots, \mathtt{Bin}_{n_1,n_2}\}$. Similarly, we have $\mathbf{Bins}_Z = \{\mathtt{Bin}_1, \ldots, \mathtt{Bin}_{n_1}\}$ and $\mathbf{Bins}_Y = \{\mathtt{Bin}_1, \ldots, \mathtt{Bin}_{n_2}\}$. To estimate $|q(Z \bowtie X \bowtie Y)|$ using the bins, we have:

$$\begin{aligned} |q(Z \bowtie X \bowtie Y)| = &|X||Y||Z| \sum_{\mathtt{Bin}_{i_1,i_2} \in \mathbf{Bins}_X} \sum_{v_1, v_2 \in D_{\mathtt{Bin}_{i_1,i_2}}} \\ & P(X.key_1 = v_1 \wedge X.key_2 = v_2 \wedge q(X)) \\ & \cdot P(Z.key_1 = v_1 \wedge q(Z)) \\ & \cdot P(Y.key_2 = v_2 \wedge q(Y)), \end{aligned} \tag{4}$$

where $v_1 \in D_{\mathtt{Bin}_{i_1} \in \mathbf{Bins}_Z}$ is satisfied for $P(Z.key_1 = v_1 \wedge q(Z))$ and $v_2 \in D_{\mathtt{Bin}_{i_2} \in \mathbf{Bins}_Y}$ is satisfied for $P(Y.key_2 = v_2 \wedge q(Y))$. Then, the task is to estimate $P(X.key_1 = v_1 \wedge X.key_2 = v_2 \wedge q(X)) \cdot P(Z.key_1 = v_1 \wedge q(Z)) \cdot P(Y.key_2 = v_2 \wedge q(Y))$ in Equation (4). To achieve this, we maintains statistics representing each bin of join keys. Formally, we define a bin, a cell of a k-dimensional grid on a table involving k groups of join keys (say $X.key_1, X.key_2$), as $B = (\mathtt{range}, \mathtt{num}, \mathtt{ndv})$,

where range is a k-tuple representing the identifier and domain of the bin B for matching bins across tables, num is the number of tuples in B, ndv is a k-tuple representing the number of distinct values of each join key in B. Besides, we have a global width decides the binning granularity, which also limits the maximum distinct values of each join key in a bin. Then, we organize bins: $\mathcal{B}_X = \{B_{1,1}, \ldots, B_{n_1,n_2}\}$ on table X, $\mathcal{B}_Z = \{B_1, \ldots, B_{n_1}\}$ on table Z and $\mathcal{B}_Y = \{B_1, \ldots, B_{n_2}\}$ on table Y.

Example 3. *The 2-dimensional grid on the right side of Figure 3(a) shows an example of constructing the aforementioned bins on table X where the global* width = 20*: the bin $B_{3,1} \in \mathcal{B}_X$, corresponding to* range *of $key_1 \in [41, 60]$, $key_2 \in [10, 29]$, has* num = 18, ndv = (9, 6)*. Similarly, we have the bin $B_3 \in \mathcal{B}_Z$ and the bin $B_1 \in \mathcal{B}_Y$. In the inference, we change the statistics by query filtering predicates to get new bins $B_{3,1} \in \mathcal{B}_{X|q(X)}, B_1 \in \mathcal{B}_{X|q(Y)}, B_3 \in \mathcal{B}_{Z|q(Z)}$ as showed on the left side of Figure 3(b). Then, we estimate the statistics after joining: $B_{3,1} \in \mathcal{B}_{Z \bowtie X \bowtie Y|q}$ (showed on the right side of Figure 3(b)) of which the $B_{3,1}$.num = 11 is the estimation to $\sum_{v_1, v_2 \in D_{\text{Bin}_{3,1}} \in \mathcal{B}_X} P(X.key_1 = v_1 \wedge X.key_2 = v_2 \wedge q(X)) \cdot P(Z.key_1 = v_1 \wedge q(Z)) \cdot P(Y.key_2 = v_2 \wedge q(Y))$. Finally, we estimate the cardinality of query q through summing* num *of all bins after filtering and joining:* $11 + 10 + 6 + 8 + 11 + 24 = 70$.

Advantages of M-QSPN. Bucket-based methods [10, 36] usually compute CardEst of tables after joining without considering the effect of base table filtering and joining among tables on the statistics on join keys (especially number of distinct values), which causes high CardEst errors. Based on QSPN models which computes accurate probabilities efficiently, M-QSPN works out the intermediate results and final results by rigorous probability calculation in each step of CardEst: base table filtering, joining estimation and decomposing complex join schema, leading to a significant reduction in multi-table CardEst errors while maintaining fast inference.

In this section, we address two challenges in the above process. Firstly, while computing statistics for a given bin B on a table is straightforward, the challenge lies in estimating $\sum_{v_1, v_2 \in B_{3,1} \in \mathcal{B}_X} P(X.key_1 = v_1 \wedge X.key_2 = v_2 \wedge q(X)) \cdot P(Z.key_1 = v_1 \wedge q(Z)) \cdot P(Y.key_2 = v_2 \wedge q(Y))$ because some tuples in the matching bins across tables are not in the joined table. Specifically, the bins of different tables, though sharing the same range, usually have different ndv. And the base table filtering predicates (e.g. $q(X)$) also change the statistics in each bin. Secondly, the aforementioned equations are not applied in join schema beyond a simple star join (e.g. $T.key_2 = S.key_2 \wedge S.key_1 = X.key_1 \wedge X.key_2 = Y.key_2 \wedge X.key_1 = Z.key_1$) since some tables are not joined directly (e.g. table T and table X) in a query. In this case, there are no value variables of join keys (say $v_1, v_2, \ldots$) that are shared by all tables. The following two subsections present our approach to addressing these challenges.

6.1 Joining Estimation with Base Table Filtering

For base table filtering, we partition each table T into subsets corresponding to join keys binning and construct a QSPN model on each table subset, like $\mathcal{X} = \{X_{1,1}, \ldots, X_{3,3}\}$ showed on the left side of Figure 3(a). And we also extract a column set A' from the column set A of each table, containing the columns used for table filtering. So we can estimate the base table filtering selectivity (probability) of each subset T_i of each table to change the statistics of bins. Then, we propose how to estimate the statistics of bins after joining.

Base Table Filtering. A QSPN model on (A', T_i) computes the single-table probability $P_{T_i}(A')$ by predicates $q(T)$. The statistics of a bin B_i are then changed by $P_{T_i}(A')$ as follows.

(1) range: It is fixed for matching bins *i.e.,* not changed.

(2) num: Naturally, B_i.num is changed to $B_i.\text{num} \cdot P_{T_i}(A')$.

(3) ndv: The change of B_i.ndv after filtering is based on a natural assumption that values of join keys are selected randomly according to single-table probability. For each join key key_j, the average frequency of each value $B_i.\text{num}/B_i.\text{ndv}_j$. Since each tuple is filtered with probability $1 - P_{T_i}(A')$, the probability that a join-key value remains after filtering is $1 - (1 - P_{T_i}(A'))^{\text{num}/\text{ndv}_j}$. Therefore, we have new $B_i.\text{ndv}_j$ after filtering as $B_i.\text{ndv}_j \cdot \left[1 - (1 - P_{T_i}(A'))^{B_i.\text{num}/B_i.\text{ndv}_j}\right]$.

Joining Estimation. After base table filtering, given a bin B_i of the center table T which covers k groups of join keys, we have a set of matching bins $\mathcal{B}^j = \{B_1, \ldots, B_{n_j}\}$ on other tables which are joined to the center table T on the join key key_j where $\forall B \in \mathcal{B}^j$, B.range $= B_i.\text{range}_j$. As Example 3 shows, the bin $B_{3,1}$ of center table X which covers 2 groups of join keys, we have $\mathcal{B}^1 = \{B_3\}$ on table Z (*i.e.,* $B_3 \in \mathcal{B}_Z$) and $\mathcal{B}^2 = \{B_1\}$ on table Y (*i.e.,* $B_1 \in \mathcal{B}_Y$). Based on this, we propose the estimation to statistics after joining.

(1) ndv: For each key_j, only values which appear in all bins of the $\mathcal{B}^j$ are in the joined table. And since there is a global width deciding the maximum distinct values in a bin, we convert the problem of estimating ndv_j after joining to estimating size of the intersection containing $B_1.\text{ndv}, \ldots, B_{n_j}.\text{ndv}$ elements. Thus, the ndv_j after joining is estimated as $\text{width} \cdot \frac{B_i.\text{ndv}_j}{\text{width}} \cdot \prod_{B \in \mathcal{B}^j} \frac{B.\text{ndv}}{\text{width}}$ which is simplified as $\frac{B_i.\text{ndv}_j}{\text{width}^{n_j}} \prod_{B \in \mathcal{B}^j} B.\text{ndv}$.

(2) num: Assuming all values of join keys appear in the joined table *i.e.,* B_i.ndv is not changed after joining, each tuple in the center table T repeats the times of average frequency of each value in a table joined to T. But this calculation causes overestimate since the B_i.ndv can be reduced a lot according to the formula of B_i.ndv above. Thus, we multiply the formula a downscale factor *scale* to take account of the effect of B_i.ndv change on the B_i.num after joining: $scale \cdot B_i.\text{num} \prod_{1 \le j \le k} \prod_{B \in \mathcal{B}^j} \frac{B.\text{num}}{B.\text{ndv}}$. And *scale* can be estimated by comparing the B_i.ndv after joining (denoted as $B_i.\text{ndv}'$) to the original B_i.ndv: $scale = \prod_{1 \le j \le k} \left(\frac{B_i.\text{ndv}'_j}{B_i.\text{ndv}_j}\right)^{\frac{1}{j}}$ where the calculation order of production is $\frac{B_i.\text{ndv}'_j}{B_i.\text{ndv}_j}$ in ascending order. The exponent $1/j$ in the formula are for the correlation among different groups of join keys.

For CardEst of queries with star join predicates (T as the center table), the result is $\sum_{B_i \in \mathcal{B}_T} B_i.\text{num}$ after filtering and joining.

6.2 CardEst for Complex Join Schema

We propose a *CardEst Graph* (showed in Figure 4) which decomposes complex join predicates into a series of star joins.

Example 4. *Given a query with join predicates: $T.key_2 = S.key_2 \wedge S.key_1 = X.key_1 \wedge X.key_2 = Y.key_2 \wedge X.key_1 = Z.key_1$, the CardEst cannot be computed by star join formula above simply since the table T and table X are not joined directly. The join graph showed in Figure 4 decomposes this query into triple star joins. First, table Y and table Z are joined to the center table X. Second, table $Z \bowtie X \bowtie Y$ are joined to the center table S. Third, table $Z \bowtie X \bowtie Y \bowtie S$ are joined to the*

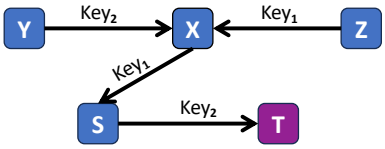


Figure 4: M-QSPN CardEst Graph.

center table T to be $Z \bowtie X \bowtie Y \bowtie S \bowtie T$. We can also change the decomposing order, such as joining T to S and joining $T \bowtie S$ to X.

This decomposition enables CardEst of such queries to be computed in the same manner as the simple star join queries.

CardEst Graph. Given a query, we construct a CardEst graph by its join predicates, of which each node v represents a table T and each edge e represents the join key between two tables. Based on this, CardEst is performed by traversing the graph in arbitrary order since the equality conditions are transitive.

When traversing the CardEst graph like Figure 4 , all edges of the graph become *directed* where a node S pointing to a node T means joining table S to T, thereby all nodes has multiple in-edge (as the center table of a star join) and only one out-edge (be joined to the center table of another star join) except the final node (e.g. the purple node T in Figure 4) which returns the CardEst result without out-edge. Once the traversal order of the CardEst graph is fixed like (Figure 4), each node v is represented by $(center, parent, child)$, where $v.center$ is its table, $v.parent = (v.parent.v, v.parent.e)$ represents the node pointing to v and the join key, and $v.child = \{(v.child_i.v, v.child_i.e)\}_{i=1}^{m}$ describes the nodes that v pointing to and the join key between v and each $v.child_i$.

The join schema is decomposed into several star joins, each processed in two steps: star join to the center table $v.center$ (bins matching, base table filtering and joining estimation); bins projection for another star join whose center table is $v.parent.v.center$ (to a 1-dimensional grid only covering the join key $v.parent.e$).

Bi-Pruning of Bins Matching. During processing star joins, some bins of $v.center$ may not matched by $v.child$ *i.e.,* the computation of their single-table probabilities is redundant. We introduce *Bi-Pruning*. Before visiting $v.child$, we prune bins of $v.center$ that are not matched by all $v.child$. After finishing calculations on $v.child$, we again prune bins of $v.center$ whose one of the matched bins of $v.child$ yields zero single-table probability.

Bins Projection is essentially merging bins that share the same $\texttt{range}_{v.parent.e}$. Given such bins $B_1, \ldots, B_n$, the merged bin B is computed as follows: (1) $B.\texttt{range} = B_i.\texttt{range}_{v.parent.e} | \forall B_i$. (2) $B.\texttt{num} = \sum_{B_i} B_i.\texttt{num}$. (3) $B.\texttt{ndv}$ is 1-tuple, recording the number of distinct values of B on the join key $v.parent.e$. We have $B.\texttt{ndv} = \texttt{width} \cdot \left[1 - \prod_{B_j} \left(1 - \frac{B_j.\texttt{ndv}_{v.parent.e}}{\texttt{width}}\right)\right]$.

7 Experiments

The source code, experimental data and the technical report which contains more detailed results are available at [45].

7.1 Experimental Setup

Datasets. We evaluate our proposed QSPN on single-table CardEst datasets and M-QSPN on multi-table CardEst datasets. Table 2 provides the statistics of the datasets.

Single-table Datasets. We use three datasets, which are commonly used in existing CardEst studies [41, 43].

- *GAS* [28] is a real-world gas sensing dataset, and we extract the most informative 8 columns (Time, Humidity, Temperature, Flow_rate, Heater_voltage, R1, R5, and R7), following the existing works [37, 43].
- *Forest* [27] is a real-world forest-fire dataset from the US Forest Service and US Geological Survey.
- *Power* [29] is an electric power consumption dataset consisting of measurements gathered in a house located in Sceaux between December 2006 and November 2010.

Multi-table Datasets. We use two multi-table datasets.

- *IMDB* [14] is a real dataset containing 50K movie reviews with a star join schema. It is extensively used in prior work for multi-table CardEst evaluation [10, 12, 36, 40, 43]. The columns in *IMDB* typically have large domain sizes, which increases the difficulty of CardEst.
- *STATS* [44] is a real dataset with 8 tables recording user-generated content, which has more complex distribution and join schema (star + chain) than *IMDB*.

Query Workloads. We describe how the query workloads used in the experiments are constructed.

Synthetic Workloads for Single-Table CardEst. Since no real query workloads are available for the three single-table datasets, we synthesize workloads as follows.

(1) *Template generation*: Following existing DBMS benchmarks [17, 31, 32], we first generate SQL templates with different column combinations as query predicates.

(2) *Template selection*: Since data correlation strongly affects CardEst performance according to prior studies [12, 16, 34, 41, 43], we group templates into two groups by accessing highly correlated columns or weakly correlated columns. We then sample templates evenly from both groups.

(3) *Query synthesis*: Given a template, we generate queries by instantiating it with randomly selected predicates. Previous studies [34, 35, 41] show that predicates significantly affect CardEst performance. Thus, following [34], we generate predicates in two steps: selecting a random tuple as the range center; determining the range width using a uniform distributio and an exponential distribution in a 1:1 mixture.

Data Updates and Workload Shifts. We generate a read-write hybrid workload where each SQL statement is randomly designated as either a query or a DML command. First, we create a training workload with queries accessing only weakly correlated columns and range constraints following a normal distribution. Then, we add at least 20% new data tuples with increased correlation by sampling from a table with sorted columns, and include the tuples as DML commands. Finally, we construct a test workload consisting of (1) queries following the original patterns and distribution and (2) queries accessing highly correlated columns with range constraints following an exponential distribution.

Real-World Workload for Multi-Table CardEst.

- For the multi-table dataset *IMDB*, we use *JOB-light* [17], a widely used multi-table workload for CardEst. It consists of 70 queries on *IMDB*, each containing star joins over 2 to 5 tables, with 1 to 5 range constraints. We use *JOB-light* as the

Table 2: Statistics of Datasets.

Dataset	Tables	Tuples	Columns (Numeric)	Domain Size
GAS	1	4M	8 (8)	$10^1 \sim 10^2$
Forest	1	0.6M	54 (10)	$10^2 \sim 10^3$
Power	1	2M	9 (7)	$10^4 \sim 10^5$
IMDB	21	74M	108 (57)	$10^0 \sim 10^7$
STATS	7	1M	43 (43)	$10^0 \sim 10^{10}$

test workload and adopt an extended workload provided by existing methods [16, 38] as the training workload.

- For the multi-table dataset *STATS*, we use *STATS-CEB* [44], containing 146 queries on *STATS*. The queries cover multiple join forms (chain, star and mixture) with varied single-table selectivities. We randomly split *STATS-CEB* into two halves as test and training workloads respectively. Both workloads contain all column patterns and join schemas.

Baselines. We compare QSPN against the following baselines.

Postgres [10] is the cardinality estimator of PostgreSQL, based on traditional statistics-based methods. We run and evaluate it using the connector from [34], which connects to PostgreSQL 12 with the default setting $stat_target = 10000$.

Sampling [34] is a sampling-based cardinality estimator whose sampling size is tuned to the same model size of QSPN.

MSCN [16] is a query-driven CardEst model based on regression models. We use its implementation from [34] and set its hyper-parameters following the configurations in [34].

Naru [41] is a DAR-based model that fits the joint data distribution for CardEst. We use the implementation of Naru and set its hyper-parameters following the configurations in [34].

NeuroCard [40] is a multi-table CardEst extension method of Naru. It trains DAR models on pre-joined tables.

DeepDB [12] is a SPN-based model that fits joint data distribution for CardEst under a local independence assumption. We use the implementation from [34] and follow its hyperparameter settings. We also apply it for multi-table CardEst.

FLAT [43] is a CardEst method based on DeepDB, which introduces factorization and multi-dimensional histograms. We use the implementation provided by the authors of FLAT [37] and tune its hyper-parameters as recommended in the original paper. We also use it as multi-table CardEst method.

UAE [35] is a hybrid-driven CardEst method based on Naru that additionally incorporates supervised losses from queries. We use the authors' single-table CardEst implementation [38].

Evaluation Metrics. We use the following metrics to comprehensively evaluate and compare CardEst methods.

Estimation Accuracy. For each query, we measure estimation accuracy using Q-error, defined as the ratio between the estimation est and the ground truth gt, *i.e.*, $Q\text{-error} = \frac{\max\{est,gt\}}{\min\{est,gt\}}$.

Inference Time. For each query in the test set, we measure the time spent for CardEst as inference time. We then compute the mean inference time among the queries in the test set.

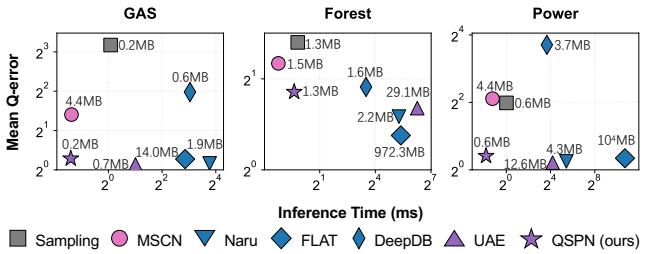


Figure 5: Pareto trade-offs among estimation accuracy, inference time, and storage overhead.

Construction Time and Storage Overhead. We measure the total time spent for constructing or learning the models as the construction time and the total size of the model or statistics for each method on each dataset as the storage overhead.

End-to-End Query Time. We measure CardEst time and query execution time. Following the benchmark setting in [44], we first compute the CardEst for all subqueries using each method to measure CardEst time. We then inject the estimated cardinalities into PostgreSQL [10] by replacing the optimizer's CardEst with the predicted values and measure the execution time of the query workload. Finally, we compare the end-to-end query time, defined as the sum of CardEst and query execution time, across all methods.

Experimental Settings. All the evaluated methods are implemented in Python 3.7. Experiments are conducted on an AMD-64 server: Ubuntu 20.04.6 LTS, 2× Intel(R) Xeon(R) Gold 6230 CPU @ 2.10GHz (20C40T), 1TB DDR4 ECC memory, 4×8 TB HDD (RAID5) Storage. We use the same default hyper-parameters of QSPN across all datasets: $\tau_p = (s = 5, l = 0.1, u = 0.3), \tau = 0.01, \tau_x = 0.7, \tau_s = 0.3$. The parameter τ_p controls the *RDC* threshold, which is set by following prior SPN settings. The parameters τ_x and τ_s correspond to the `QProduct` and `QSplit` algorithms and are selected via a grid search on workload samples. For multi-table CardEst experiments, we set $width$ = 1,500,000 for *IMDB* and $width$ = 60,000 for *STATS* based on the join-key domain sizes.

7.2 Evaluation on Single-Table CardEst

This section evaluates QSPN on the single-table CardEst datasets. We present a Pareto analysis to compare the overall trade-offs among estimation accuracy, inference time, and storage overhead, and detailed comparisons on each criterion.

Pareto Trade-off Analysis. Figure 5 presents Pareto comparison of all methods on the three single-table datasets with respect to estimation accuracy, inference time, and storage overhead. Each method is represented as a point in the two-dimensional space defined by mean Q-error and inference time, with storage overhead annotated for each point.

QSPN lies on the *Pareto frontier* across the evaluated datasets, balancing estimation accuracy, inference time and storage overhead. For example, on *GAS*, QSPN achieves a 1.22 mean Q-error, 0.379 ms inference, 0.209 MB storage overhead, with no baseline improving all three metrics. It matches comparable estimation accuracy of data-driven *FLAT* and *Naru* at lower inference time and storage overhead, whereas query-driven *MSCN* trades accuracy for faster inference, traditional sampling methods trade accuracy and inference time for

storage, and SPN-based model *DeepDB* offers a weaker accuracy–latency trade-off on highly correlated data.

Individual Metric Analysis.

Estimation Accuracy. QSPN achieves mean Q-errors of 1.0–1.8, within 10–15% of DAR-based *UAE*, while generally outperforming query-driven *MSCN* and sampling-based *Sampling*. QSPN reduces Q-error by 20–93% over *MSCN* and by up to 90% over *DeepDB* on *Power*, owing to its column partitioning which considers both data distribution and query co-access patterns.

Inference Time. QSPN achieves 0.2–0.8 ms fast inference across all datasets, over 10× faster than data-driven models. *Naru* and *UAE* are slowest due to progressive sampling. QSPN also reduces inference time by about 95% over *DeepDB*, as its column partitioning produces smaller SPNs. In contrast, *FLAT* is slower on *Forest* and *Power* because modeling complex correlations requires more factorized and multi-leaf nodes.

Storage Overhead. QSPN is the most lightweight learned CardEst model (tens of KB to about 1 MB). *DeepDB* is also compact but grows on *Power* due to more nodes. CNN/DAR-based *MSCN*, *Naru* and *UAE* are larger, while *FLAT* incurs substantial storage overhead from the same structural complexity that slows its inference.

Construction Time. Construction time is negligible for *Postgres* and *Sampling*, minutes for query-driven *MSCN* and SPN-based *DeepDB*, *FLAT* and QSPN, and hours for DAR-based *Naru* and *UAE*. The detailed results are in the techinical report [45] due to space constraints.

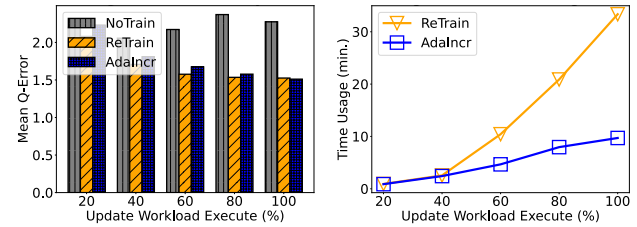
7.3 Evaluation on Dynamic Model Update

We evaluate the model update performance of QSPN by comparing the following alternatives: *NoTrain* (no model updates), *ReTrain* (periodic model reconstruction), and *AdaIncr* (our updating method in QSPN) in Figure 6, which involves both data updates and workload shifts *i.e.,* difficult case for model update.

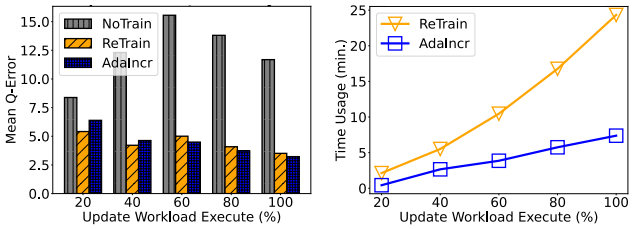
NoTrain suffers from poor accuracy because its stale model cannot adapt to evolving data and workloads. In contrast, both *ReTrain* and *AdaIncr* preserve the estimation accuracy of QSPN, demonstrating their ability to capture these changes effectively. Notably, *AdaIncr* reduces update time by 30%–60% while reaching the comparable accuracy of *ReTrain*. As the data and workload grow, the cumulative update time of *AdaIncr* increases gradually, whereas that of *ReTrain* rises sharply. This gap occurs because *ReTrain* rebuilds the entire model over increasingly large datasets, making each update more expensive. By continuously incorporating changes, *AdaIncr* processes only a small data batch and modifies only a few affected nodes each time, thereby avoiding unnecessary reconstruction of unchanged model components. This incremental strategy keeps model updates lightweight and improves scalability as the data expands with workload shifts.

7.4 Evaluation of Multi-Table CardEst

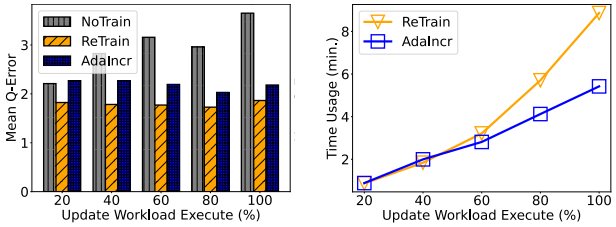
In this section, we evaluate multi-table CardEst and report the results on two datasets and workloads, *JOB-light* and *STATS-CEB*, in Table 3. We compare M-QSPN with *DeepDB*, *FLAT*, *NeuroCard*, and compute their evaluation metrics based on the execution results of an existing benchmark [21].



(a) Dataset GAS.



(b) Dataset Forest.



(c) Dataset Power.

Figure 6: Accuracy and Time Usage on Hybrid-Workload.

First, M-QSPN achieves the best average estimation accuracy across the datasets. On *JOB-light*, all methods obtains comparable Q-errors due to the simple join schema and predicate patterns. On more complex *STATS-CEB*, M-QSPN achieves the lowest mean Q-error by a significant margin. Baselines are trained on samples of pre-joined tables, reducing accuracy. This is due to the massive pre-joined tables according to the complex join schema. In contrast, M-QSPN models tables independently and is less sensitive to join-graph complexity.

Second, M-QSPN achieves the lowest inference time on both datasets, 4×–40× faster than the baselines on *STATS-CEB* in particular. In spite of being faster than the slowest *NeuroCard*, *DeepDB* and *FLAT* require 87 and 175 ms large inference time because their SPNs over pre-joined tables with exponentially growing size contain more nodes and layers. By modeling each table independently, M-QSPN keeps inference cost low which exhibits linear growth with the number of joined tables.

Third, M-QSPN offers competitive construction time and storage by modeling each table independently and performing join reasoning at query time, avoiding model construction on large pre-joined tables. Its model is larger on *JOB-Light* because duplicated non-join-key values across table subsets enlarge leaf histograms. Replacing histograms with lightweight piecewise linear models (PLMs) could further reduce storage.

Table 3: Evaluating Multi-Table CardEst.

Dataset & Workload	Method	Estimation Errors						Inference Time (ms)	Construction Time (min.)	Model Size (MB)
		50th	90th	95th	99th	Max	Mean			
JOB-light	DeepDB	1.69	2.79	**3.71**	15.53	27.1	**2.24**	44	59	34
	FLAT	**1.40**	**2.68**	5.41	22.65	33.00	2.46	9	**3**	51
	NeuroCard	1.65	4.39	6.97	**10.8**	**14.52**	2.51	608	60	**7**
	M-QSPN	1.50	4.66	6.67	12.27	16.07	2.32	**4**	6	136
STATS-CEB	DeepDB	**2.28**	69.44	213.26	22471.59	78445.37	1103.49	87	108	162
	FLAT	3.69	39.10	127.88	22571.58	79765.19	1112.70	175	262	310
	NeuroCard	1194.60	$1.2 * 10^7$	$2.4 * 10^8$	$8.0 * 10^8$	$1.4 * 10^9$	$3.9 * 10^7$	823	568	337
	M-QSPN	2.93	**12.80**	**25.15**	**49.48**	**52.41**	**6.09**	**19**	**3**	**28**

Table 4: Evaluating End-to-End Performance.

Benchmark	Method	CardEst Time (s)	Query Exec. Time(s)	Total Time Saved
JOB-light	PostgreSQL	/	4122	/
	DeepDB	31	4035	1.4%
	FLAT	6	4087	0.7%
	NeuroCard	423	**3544**	3.8%
	M-QSPN	**3**	3606	**12.4%**
STATS-CEB	PostgreSQL	/	11613	/
	DeepDB	226	10683	6.1%
	FLAT	460	10304	7.3%
	NeuroCard	2142	12799	-28.7%
	M-QSPN	**49**	**10271**	**11.1%**

7.5 Evaluation on End-to-End Query Execution

Based on the multi-table CardEst results, we evaluate the effect of CardEst models on end-to-end query execution in PostgreSQL [10] using the settings described in Section 7.1. We run the end-to-end benchmark on *JOB-light* and *STATS-CEB*, comparing M-QSPN with the PostgreSQL internal estimator [10] and learned baselines.

The results are reported in Table 4. In general, learned CardEst methods improve the effectiveness of query optimization in most cases, compared with the original PostgreSQL estimator. In particular, M-QSPN reduces end-to-end query total time by 12.4% on *JOB-light* and 11.1% on *STATS-CEB*, both superior to all baselines.

On the *JOB-light* dataset, all learned CardEst methods reduces query execution time. This is because they are all accurate on this dataset. Comparing query execution time, they can be categorized into two groups: (1) *DeepDB* and *FLAT* (Around 4000 seconds); (2) *NeuroCard* and M-QSPN (Around 3500 seconds). This is due to *NeuroCard* and M-QSPN achieving much lower 99th and Max Q-error, making CardEst quality more stable. Although *NeuroCard* brings even shorter query execution time than M-QSPN, it is too slow in inference to achieve overall efficient (only 3.8% total time saved). Thus, the fast M-QSPN saved the most total time (over 10%).

On the more complex *STATS-CEB* workload, *NeuroCard* fails to optimize query execution (-28.7% total time saved) because it delivers very high CardEst errors, while other baselines and M-QSPN all produce more accurate cardinalities than the PostgreSQL estimator, resulting in better join orders and access paths and lower execution time. Since M-QSPN is much faster and more accurate in CardEst on *STATS-CEB*, it achieves the least CardEst time and query execution time on this benchmark. Thus, M-QSPN is completely ahead of baselines, saving total query time more than 10%.

8 Related Work

Traditional CardEst Methods. Postgres [10] relies on the assumption of column independence and estimates the data distribution using histograms. Sampling-based methods [2, 11, 18] execute queries on the stored data sample for CardEst. MHist [24] constructs multi-dimensional histograms for correlated data. Bayes [4] performs CardEst using probabilistic graphical models [5, 8, 33]. The key limitation of traditional methods is their simplifying assumptions, leading to significant errors.

Learning-based CardEst Methods. Query-driven methods transform the CardEst problem into a regression task, mapping query workloads to ground truth cardinalities. MSCN [16] fed a standardized vector which encodes a query into a learning framework of four Multi-Layer Perceptrons (MLPs) whose output is the selectivity. LW-XGB/NN [6] encodes a query by concatenating all range predicates in order. This query vector is then fed into a small XGBoost [3] model or NN for CardEst. Data-driven methods transform the CardEst problem into a joint probability problem, where each column is treated as a random variable. Naru [41] factorizes the joint distribution into conditional distributions using DAR models such as MADE [7] and then employs progressive sampling for CardEst. UAE [35] is an optimized version of Naru, which is a hybrid model to fits data with long-tail distributions better by adjusting the sampling regions according to the query workload. FactorJoin [36] is a multi-table CardEst method that uses buckets of join keys for multi-table CardEst. Despite advancements in learning-based CardEst, existing methods may struggle to simultaneously optimize the three key criteria.

9 Conclusion

In this paper, we have introduced QSPN, a *unified model* that integrates both data distribution and query workload. QSPN extends the simple yet effective Sum-Product Network (SPN) model by jointly partitioning columns based on both data correlations and query access patterns, thereby reducing model size and improving inference efficiency without sacrificing estimation accuracy. We have formalized QSPN as a tree-based structure that extends SPNs by introducing two new node types: QProduct and QSplit. We have also proposed a multi-table CardEst method M-QSPN by extending QSPN. We have conducted extensive experiments to evaluate QSPN and M-QSPN. The experimental results have demonstrated that they achieves superior and robust performance on the three key criteria, compared with state-of-the-art approaches.

References

[1] Moses Charikar, Surajit Chaudhuri, Rajeev Motwani, and Vivek Narasayya. 2000. Towards estimation error guarantees for distinct values. In *Proceedings of the nineteenth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. 268–279.

[2] Surajit Chaudhuri, Gautam Das, and Utkarsh Srivastava. 2004. Effective Use of Block-Level Sampling in Statistics Estimation. In *Proceedings of the 2004 ACM SIGMOD international conference on Management of data*. 287–298.

[3] Tianqi Chen and Carlos Guestrin. 2016. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*. 785–794.

[4] CKCN Chow and Cong Liu. 1968. Approximating discrete probability distributions with dependence trees. *IEEE transactions on Information Theory* 14, 3 (1968), 462–467.

[5] Amol Deshpande, Minos Garofalakis, and Rajeev Rastogi. 2001. Independence is good: Dependency-based histogram synopses for high-dimensional data. *ACM SIGMOD Record* 30, 2 (2001), 199–210.

[6] Anshuman Dutt, Chi Wang, Azade Nazi, Srikanth Kandula, Vivek Narasayya, and Surajit Chaudhuri. 2019. Selectivity estimation for range predicates using lightweight models. *Proceedings of the VLDB Endowment* 12, 9 (2019), 1044–1057.

[7] Mathieu Germain, Karol Gregor, Iain Murray, and Hugo Larochelle. 2015. MADE: masked autoencoder for distribution estimation. In *Proceedings of the 32nd International Conference on International Conference on Machine Learning-Volume 37*. 881–889.

[8] Lise Getoor, Benjamin Taskar, and Daphne Koller. 2001. Selectivity estimation using probabilistic models. In *Proceedings of the 2001 ACM SIGMOD international conference on Management of data*. 461–472.

[9] Goetz Graefe and William J McKenna. 1993. The volcano optimizer generator: Extensibility and efficient search. In *Proceedings of IEEE 9th international conference on data engineering*. IEEE, 209–218.

[10] Copyright © 1996-2024 The PostgreSQL Global Development Group. [n.d.]. PostgreSQL 12.20 Documentation: Chapter 59. Genetic Query Optimizer. ([n. d.]). https://www.postgresql.org/docs/12/geqo-intro2.html

[11] Peter J Haas, Jeffrey F Naughton, S Seshadri, and Lynne Stokes. 1995. Sampling-based estimation of the number of distinct values of an attribute. In *VLDB*, Vol. 95. 311–322.

[12] Benjamin Hilprecht, Andreas Schmidt, Moritz Kulessa, Alejandro Molina, Kristian Kersting, and Carsten Binnig. 2020. DeepDB: learn from data, not from queries! *Proceedings of the VLDB Endowment* 13, 7 (2020), 992–1005.

[13] Tsuyoshi Hirayama, Yuhao Liu, Kazuhisa Makino, Ke Shi, and Chao Xu. 2023. A Polynomial Time Algorithm for Finding a Minimum 4-Partition of a Submodular Function. (2023), 1680–1691. https://doi.org/10.1137/1.9781611977554.CH64

[14] Research institute for mathematics & computer science in the Netherlands. [n.d.]. imdb.tgz. ([n. d.]). https://homepages.cwi.nl/~boncz/job/imdb.tgz

[15] Yannis E Ioannidis and Stavros Christodoulakis. 1991. On the propagation of errors in the size of join results. In *Proceedings of the 1991 ACM SIGMOD International Conference on Management of data*. 268–277.

[16] Andreas Kipf, Thomas Kipf, Bernhard Radke, Viktor Leis, Peter A. Boncz, and Alfons Kemper. 2019. Learned Cardinalities: Estimating Correlated Joins with Deep Learning. In *9th Biennial Conference on Innovative Data Systems Research, CIDR 2019, Asilomar, CA, USA, January 13-16, 2019, Online Proceedings*. www.cidrdb.org. http://cidrdb.org/cidr2019/papers/p101-kipf-cidr19.pdf

[17] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2015. How good are query optimizers, really? *Proceedings of the VLDB Endowment* 9, 3 (2015), 204–215.

[18] Viktor Leis, Bernhard Radke, Andrey Gubichev, Alfons Kemper, and Thomas Neumann. 2017. Cardinality Estimation Done Right: Index-Based Join Sampling.. In *Cidr*.

[19] David López-Paz, Philipp Hennig, and Bernhard Schölkopf. 2013. The Randomized Dependence Coefficient. In *Advances in Neural Information Processing Systems 26: 27th Annual Conference on Neural Information Processing Systems 2013. Proceedings of a meeting held December 5-8, 2013, Lake Tahoe, Nevada, United States*, Christopher J. C. Burges, Léon Bottou, Zoubin Ghahramani, and Kilian Q. Weinberger (Eds.). 1–9. https://proceedings.neurips.cc/paper/2013/hash/aab3238922bcc25a6f606eb525ffdc56-Abstract.html

[20] Alejandro Molina, Antonio Vergari, Nicola Di Mauro, Sriraam Natarajan, Floriana Esposito, and Kristian Kersting. 2018. Mixed Sum-Product Networks: A Deep Architecture for Hybrid Domains. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence, (AAAI-18), the 30th innovative Applications of Artificial Intelligence (IAAI-18), and the 8th AAAI Symposium on Educational Advances in Artificial Intelligence (EAAI-18), New Orleans, Louisiana, USA, February 2-7, 2018*, Sheila A. McIlraith and Kilian Q. Weinberger (Eds.). AAAI Press, 3828–3835. https://doi.org/10.1609/AAAI.V32I1.11731

[21] Nathaniel-Han. [n.d.]. End-to-End-CardEst-Benchmark. ([n. d.]). https://github.com/Nathaniel-Han/End-to-End-CardEst-Benchmark

[22] Kazumasa Okumoto, Takuro Fukunaga, and Hiroshi Nagamochi. 2009. Divide-and-Conquer Algorithms for Partitioning Hypergraphs and Submodular Systems. 5878 (2009), 55–64. https://doi.org/10.1007/978-3-642-10631-6_8

[23] Hoifung Poon and Pedro Domingos. 2011. Sum-product networks: A new deep architecture. In *2011 IEEE International Conference on Computer Vision Workshops (ICCV Workshops)*. IEEE, 689–690.

[24] Viswanath Poosala and Yannis E Ioannidis. 1997. Selectivity estimation without the attribute value independence assumption. In *VLDB*, Vol. 97. 486–495.

[25] Foster Provost, David Jensen, and Tim Oates. 1999. Efficient progressive sampling. In *Proceedings of the fifth ACM SIGKDD international conference on Knowledge discovery and data mining*. 23–32.

[26] UCI ML Repository. [n.d.]. Census Income Data Set. ([n. d.]). https://archive.ics.uci.edu/dataset/20/census+income

[27] UCI ML Repository. [n.d.]. Classification of pixels into 7 forest cover types. ([n. d.]). https://archive.ics.uci.edu/dataset/31/covertype

[28] UCI ML Repository. [n.d.]. Gas sensor array temperature modulation Data Set. ([n. d.]). https://archive.ics.uci.edu/ml/datasets/Gas+sensor+array+temperature+modulation

[29] UCI ML Repository. [n.d.]. Individual Household Electric Power Consumption. ([n. d.]). https://archive.ics.uci.edu/dataset/235/individual+household+electric+power+consumption

[30] Mechthild Stoer and Frank Wagner. 1997. A simple min-cut algorithm. *J. ACM* 44, 4 (1997), 585–591. https://doi.org/10.1145/263867.263872

[31] TPC. [n.d.]. TPC-DS. ([n. d.]). https://www.tpc.org/tpcds/default5.asp

[32] TPC. [n.d.]. TPC-H. ([n. d.]). https://www.tpc.org/tpch/default5.asp

[33] Kostas Tzoumas, Amol Deshpande, and Christian S Jensen. 2011. Lightweight graphical models for selectivity estimation without independence assumptions. *Proceedings of the VLDB Endowment* 4, 11 (2011), 852–863.

[34] Xiaoying Wang, Changbo Qu, Weiyuan Wu, Jiannan Wang, and Qingqing Zhou. 2021. Are we ready for learned cardinality estimation? *Proceedings of the VLDB Endowment* 14, 9 (2021), 1640–1654.

[35] Peizhi Wu and Gao Cong. 2021. A unified deep model of learning from both data and queries for cardinality estimation. In *Proceedings of the 2021 International Conference on Management of Data*. 2009–2022.

[36] Ziniu Wu, Parimarjan Negi, Mohammad Alizadeh, Tim Kraska, and Samuel Madden. 2023. FactorJoin: a new cardinality estimation framework for join queries. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–27.

[37] wuziniu. 2022. Github repository: FSPN. (2022). https://github.com/wuziniu/FSPN

[38] Jingyi Yang. [n.d.]. Github repository: pagegitss/UAE. ([n. d.]). https://github.com/pagegitss/UAE

[39] Jingyi Yang. [n.d.]. Github repository: pagegitss/UAE. ([n. d.]). https://github.com/pagegitss/UAE/tree/master/UAE_joins

[40] Zongheng Yang, Amog Kamsetty, Sifei Luan, Eric Liang, Yan Duan, Xi Chen, and Ion Stoica. 2020. NeuroCard: one cardinality estimator for all tables. *Proceedings of the VLDB Endowment* 14, 1 (2020), 61–73.

[41] Zongheng Yang, Eric Liang, Amog Kamsetty, Chenggang Wu, Yan Duan, Xi Chen, Pieter Abbeel, Joseph M. Hellerstein, Sanjay Krishnan, and Ion Stoica. 2019. Deep Unsupervised Cardinality Estimation. *Proc. VLDB Endow.* 13, 3 (2019), 279–292. https://doi.org/10.14778/3368289.3368294

[43] Rong Zhu, Ziniu Wu, Yuxing Han, Kai Zeng, Andreas Pfadler, Zhengping Qian, Jingren Zhou, and Bin Cui. 2021. FLAT: fast, lightweight and accurate method for cardinality estimation. *Proceedings of the VLDB Endowment* 14, 9 (2021), 1489–1502.

[43] Rong Zhu, Ziniu Wu, Yuxing Han, Kai Zeng, Andreas Pfadler, Zhengping Qian, Jingren Zhou, and Bin Cui. 2021. FLAT: fast, lightweight and accurate method for cardinality estimation. *Proceedings of the VLDB Endowment* 14, 9 (2021), 1489–1502.

[44] Yuxing Han, Ziniu Wu, Peizhi Wu, Rong Zhu, Jingyi Yang, Liang Wei Tan, Kai Zeng, Gao Cong, Yanzhao Qin, Andreas Pfadler, Zhengping Qian, Jingren Zhou, Jiangneng Li, and Bin Cui. 2021. Cardinality Estimation in DBMS: A Comprehensive Benchmark Evaluation. *Proceedings of the VLDB Endowment* 15, 4 (2021), 752–765.

[45] QSPN_code. https://anonymous.4open.science/r/QSPN_code-1FCC

A Additional Experimental Results

We show the whole experimental results of single-table CardEst in Table 5.

B Additional Analysis

B.1 Complexity Analysis of Offline QSPN Construction

Given data T, workload Q and column set $|\mathcal{A}| = m$, supposing a balanced c-way QSPN tree with n nodes (s Sum nodes, x QSplit nodes, u Product | QProduct nodes, less than and around n Leaf nodes) and z sample size for RDC. For each Leaf node, the number of tuples is around $|T|/s$ so its complexity is $O(n|T|/s)$. For each of the data tuples, it will go through $log_c s$ Sum nodes and cost $O(cm)$ in each node so the complexity of Sum nodes is $O(cm|T|log_c s)$. Similarly, the complexity of QSplit nodes is $O(cm|Q|log_c t)$. The cases are different with Product and QProduct, since their cost contains RDC and AFF matrixes computation. For RDC matrix computation, the cost is constant $O(zlogz)$. So the complexity of Product nodes is $O(nm^2 zlogz)$. For AFF matrix calculation, the cost is $O(m^2|Q|)$. So the complexity of QProduct nodes is $O((s+x+u)m^2|Q|)$. In total, the complexity of QSPN construction is $O(m^2[nzlogz+(s+x+u)|Q|]+cm(|T|log_c s+|Q|log_c x)+n|T|/s)$.

B.2 Complexity Analysis of Online QSPN Inference

Given a query q with selectivity (probability) p and constraints on m columns, suppose the QSPN model is a balanced c-way tree with N nodes (s Sum nodes, x QSplit nodes, and u Product/QProduct nodes). It is known that the cost of a Leaf node with a histogram is $O(1)$, the cost of a QSplit node is $O(m^2 c)$, and the cost of other middle nodes is $O(mc)$. Thus, the worst-case complexity of the bottom-up inference method is $O(xm^2 c+(N-x)mc)$. Since there are usually a small number of QSplit nodes in a QSPN model for query routing, the time cost of our fast inference method is approximately $\max(p, \frac{1}{s}) \cdot \frac{m}{u} \cdot Nmc$.

C Additional Detailed Algorithm

C.1 Offline QSPN Construction Algorithm

We show our top-down **ConstructQSPN** (Algorithm 1) corresponding to Section 4 including its sub function **PartitionByAFF** (Algorithm 1.1) which is explained in Section 4.1 and sub function **SplitWorkload** (Algorithm 1.2) which is explained in Section 4.2. In **ConstructQSPN** algorithm, each node n to construct are tried different type n.O in the heuristic order: Leaf, Product, QProduct, QSplit, Sum for a accuracy-efficiency balanced QSPN model. And the specific node construction are executed in sub functions.

C.2 Online QSPN Inference Algorithm

We show our top-down **QSPN-Online** (Algorithm 2) based on query routing and pruning rules whose sub function **RouteQuery** is explained in Section 5.1. Benefiting from sufficient optimization utilizing the features of QSPN, the **QSPN-Online** algorithm provides accurate and fast CardEst in online phase using lightweight QSPN model which are constructed both on data and workload before.

Algorithm 1 **Construct**QSPN (A, T, Q)

Input: A: A Column Set; T: A Table; Q: A Query Workload
Output: n: A QSPN Node

Initialize a QSPN node n = (A, T, Q, O)
if $|A| = 1$ **then**
  n.$O \leftarrow$ Leaf
  n.$Pr_T(A) \leftarrow$ **BuildHist** (A, T)
else if $\mathcal{A} \leftarrow$ **PartitionByRDC** (A, T) is *non-singleton* **then**
  n.$O \leftarrow$ Product
  n.child$_i \leftarrow$ **Construct**QSPN (A_i, T, Q) for $\forall A_i \in \mathcal{A}$
else if $\mathcal{A} \leftarrow$ **PartitionByAFF** (A, Q) is *non-singleton* **then**
  n.$O \leftarrow$ Product
  n.child$_i \leftarrow$ **Construct**QSPN (A_i, T, Q) for $\forall A_i \in \mathcal{A}$
else if $\mathcal{Q} \leftarrow$ **SplitWorkload** (Q, A) is *non-singleton* **then**
  n.$O \leftarrow$ QSplit
  n.child$_i \leftarrow$ **Construct**QSPN (A, T, Q_i) for each $Q_i \in \mathcal{Q}$
else
  n.$O \leftarrow$ Sum
  $\mathcal{T} \leftarrow$ **ClusterTable** (T, A)
  Compute weight $w_i \leftarrow |T_i|/|T|$ for each sub-table $T_i \in \mathcal{T}$
  n.child$_i \leftarrow$ **Construct**QSPN (A, T_i, Q) for each $T_i \in \mathcal{T}$
end if
return node n

Algorithm 1.1 **PartitionByAFF** (Q, A)

Input: Q: A Query Workload; A: A Column Set
Output: $\mathcal{A} = \{A_1, \ldots, A_m\}$: A Collection of Column Sets

for any pair (a_i, a_j) from A **do**
  Compute AFF$(a_i, a_j|Q)$ for a_i and a_j
end for
Construct a graph $G = (A, E)$ where vertices A represent columns, and an edge $e_{ij} \in E$ exists between columns a_i and a_j if their affinity AFF$(a_i, a_j|Q)$ is larger than a threshold τ.
$\mathcal{G} \leftarrow$ **ConnectedComponents** (G)
return $\mathcal{A} = \{A_i \mid$ vertex set in each component $G_i \in \mathcal{G}\}$

Algorithm 2 **QSPN-Online** (n, q)

Input: n: A QSPN Node; q: A Query
Output: P_{n}: Probability Estimated at Node n

if n.O = Leaf **then**
  return n.$P_T(q)$ based on the histogram at leaf n
else if n.O = Product | QProduct **then**
  $P_i \leftarrow$ **QSPN-Online** (n.child$_i, q$) for each child of n with the columns satisfying $q.A \cap$ n.child$_i.A \neq \emptyset$
  return $\prod_i P_i$ from all child nodes of n
else if n.O = QSplit **then**
  $k \leftarrow$ **RouteQuery** $(q, \{\text{n.child}_i\})$
  return QSPN-Online (n.child$_k, q$)
else
  $C \leftarrow$ **SelectChildren** $(\{\text{n.child}_i\}, q)$
  $P_i \leftarrow$ **QSPN-Online** (child$, q$) for each child $\in C$
  return weighted summation $\sum_i w_i \cdot P_i$
end if

Table 5: Evaluating Single-Table CardEst on Key Criteria: Estimation Errors, Inference/Construction Time and Model Size.

Dataset	Method	Estimation Errors						Inference Time (ms)	Construction Time (min.)	Model Size (MB)
		50th	90th	95th	99th	Max	Mean			
GAS	Postgres	3.42	33.37	48.46	143.34	376.21	13.96	**0.321**	0.123	**0.011**
	Sampling	1.06	1.42	1.87	240.88	1209.00	9.03	1.061	**0.004**	0.210
	MSCN	1.46	3.61	6.45	22.21	43.96	2.65	0.386	0.157	4.423
	Naru	1.07	1.26	1.39	1.87	2.47	1.12	13.658	149.205	1.923
	DeepDB	1.06	9.32	14.22	48.23	83.05	3.94	8.220	3.316	0.580
	FLAT	**1.02**	1.70	2.12	3.12	3.94	1.21	7.251	0.810	13.996
	UAE	1.06	**1.21**	**1.30**	**1.71**	**2.14**	**1.10**	2.013	301.390	0.747
	QSPN	1.06	1.69	2.12	3.12	3.94	1.22	0.379	1.919	0.209
Forest	Postgres	3.43	40.67	109.74	1002.98	9215.50	64.72	**0.153**	0.158	**0.071**
	Sampling	1.07	**1.44**	1.89	25.01	169.00	2.64	0.881	**0.001**	1.261
	MSCN	1.49	3.53	5.36	11.73	61.50	2.25	0.427	0.128	1.501
	Naru	1.17	1.79	2.31	7.39	**22.00**	1.50	38.651	80.026	2.216
	DeepDB	1.07	1.57	2.02	4.81	148.00	1.88	11.295	2.562	1.640
	FLAT	**1.02**	1.48	**1.80**	3.67	38.71	**1.30**	41.472	1.821	972.307
	UAE	1.16	1.82	2.75	12.96	25.20	1.60	76.721	188.661	29.147
	QSPN	1.05	1.48	1.84	**3.08**	186.50	1.81	0.770	2.167	1.261
Power	Postgres	3.71	162.04	925.33	13374.34	48341.00	551.04	**0.103**	0.206	**0.041**
	Sampling	1.04	1.39	3.88	60.96	394.00	3.96	1.011	**0.001**	0.624
	MSCN	1.88	8.31	15.58	32.45	155.00	4.32	0.421	0.144	4.402
	Naru	1.06	1.35	1.61	3.48	7.95	1.19	43.125	175.368	4.327
	DeepDB	1.04	3.42	5.45	198.20	1876.00	13.05	12.731	3.722	3.711
	FLAT	**1.01**	**1.22**	**1.39**	**2.27**	50.59	1.27	1694.277	2.225	10100.460
	UAE	1.04	1.27	1.58	4.15	**5.00**	**1.16**	18.220	1316.239	12.599
	QSPN	1.05	1.73	2.43	6.33	12.60	1.33	0.281	0.782	0.623

Algorithm 3 `QSPN-Update` $(\mathsf{n}, \Delta T, \Delta Q)$

Input: n: A QSPN Node, ΔT: Data Update subset, ΔQ: New Queries subset
Output: Updated QSPN node n'

$U \leftarrow False$
if $\mathsf{n}.O = \texttt{Leaf}$ **then**
  $\mathsf{n}' \leftarrow \mathsf{n}$
  Modify Histogram at n' by ΔT
  return n'
else if $\mathsf{n}.O = \texttt{Product} \mid \texttt{QProduct}$ **then**
  $U, \mathsf{n}' \leftarrow$ **ColumnPartitionCheck** $(\mathsf{n}, \Delta T, \Delta Q)$
else if $\mathsf{n}.O = \texttt{QSplit}$ **then**
  $U, \mathsf{n}' \leftarrow$ **WorkloadSplitCheck** $(\mathsf{n}, \Delta Q)$
else if $\mathsf{n}.O = \texttt{Sum}$ **then**
  $U, \mathsf{n}' \leftarrow$ **DataPartitionCheck** $(\mathsf{n}, \Delta T)$
end if
if U **then**
  $\mathsf{n}' \leftarrow$ **QSPN-Offline** $(\mathsf{n}'.A, \mathsf{n}'.T \cup \Delta T, \mathsf{n}'.Q \cup \Delta Q)$
else
  $\mathsf{n}'.\texttt{child}_i \leftarrow$ **QSPN-Update** $(\mathsf{n}'.\texttt{child}_i, \Delta T_i, \Delta Q_i)$ for each child of n'
end if
return n'

Algorithm 1.2 `SplitWorkload` (Q, A)

Input: Q: A Query Workload; A: A Column Set
Output: $\mathcal{Q} = \{Q_1, \ldots Q_m\}$: A Collection of Sub-workloads

$\mathbf{q_1}, \ldots, \mathbf{q_w} \leftarrow$ Merge Queries q sharing the same $\mathbf{acc}_q$
$\texttt{sim} \leftarrow$ **ComputeSim** $(\mathbf{ACC}(Q, A))$ for each (q_i, q_j) from Q
Construct a graph $G = (V, E)$ where vertice V represents $\mathbf{ACC}(Q, A)_1, \ldots, \mathbf{ACC}(Q, A)_{|\mathbf{ACC}(Q,A)|}$ and $e_{kl} \in E$ represents $\texttt{sim}(\mathbf{ACC}(Q, A)_k, \mathbf{ACC}(Q, A)_l)$
Cut off some biggests edges of G
Sort $v \in V$ by weighted degrees in desc order
Create m empty set: $R_1, \ldots, R_m$
for $1 \le i \le |\mathbf{ACC}(Q, A)|$ **do**
  Put $\mathbf{ACC}(Q, A)_i$ into the subset R_j with $\min_j(\sum_{\mathbf{ACC}(Q,A)_i \in R_j} \texttt{sim}(i, \mathbf{ACC}(Q, A)_k))$
end for
Create m empty query subsets: $Q_1, \ldots, Q_m$
for $q \in Q$ **do**
  Put q into Q_k if $\exists \mathbf{ACC}(Q, A)_l \in R_k$ and q obeys $\mathbf{ACC}(Q, A)_l$
end for
$\mathcal{Q} \leftarrow \{Q_1, \ldots, Q_m\}$
return $\mathcal{Q}$

C.3 Online QSPN Update Algorithm

We show our efficient top-down **QSPN-Update** (Algorithm 3) based on node examining whose sub function **ColumnPartitionCheck**, **WorkloadSplitCheck** and **DataPartitionCheck** are explained in Section 5.2. The **QSPN-Update** algorithm achieves adaptive incremental QSPN update to keep the model accurate against data update and query workload shift in online phase of dynamic scenes, which is manual-triggering-free and brings little extra cost.